QUANTUMBRIDGE

Official Research Log

Quantum Chip Emulator Platform — Phase 1: Foundations

Author: Darknight (Mirr) | BTech AI/ML
Affiliation: Lachoo Memorial College, Jodhpur ()

Research Start Date: July 2026

About this document

This is the official QuantumBridge research log. Every experiment, setup step, observation, and decision made during this project is recorded here with full context — what was done, why it was done, how it works, and what was observed. This document will grow entry by entry throughout the project and will form the basis of any future research publication or report.

It is written so that any researcher reading it cold can understand every decision made.

Project summary

QuantumBridge is a machine learning-based quantum chip emulator platform. It collects experimental data from real quantum hardware (IBM, Google), trains ML models to learn each chip's unique noise patterns and behavior, and packages those models as downloadable software — enabling faithful offline quantum simulation on consumer laptops.

Think: Ollama, but for quantum computers instead of LLMs.

Target market: Indian engineering colleges, small research teams, individual developers.

How to read this log

Each entry covers one experiment or setup milestone. Every entry contains:

1. What was done | 2. Why it was necessary | 3. How it works (theory) | 4. Results observed

5. What it means for the project | 6. Next step triggered by this entry

Research Log — Table of Contents

Entry	Title	Date	Status
Entry 001	Environment Setup — Python 3.11 + Qiskit Installation	July 11, 2026	Complete
Entry 002	Experiment 1 — Bell State Circuit on Aer Simulator	July 11, 2026	Complete
Entry 003	Experiment 2 — Bell State on Real IBM Hardware (ibm_fez)	July 13, 2026	Complete
Entry 004	Noise Dataset Collection — 20-Run Study (ibm_fez)	July 14, 2026	Complete
Entry 005	Multi-Circuit Dataset & Noise Scaling (80 runs)	July 14, 2026	Complete
Entry 006	CNOT-Scaling Test — 4-5 Qubits, Cross-Chip Finding	July 15, 2026	Complete
Entry 007	First Predictive Model — R2=0.747 (60 samples)	July 15, 2026	Complete
Entry 008	Single-Qubit Gate Cost & Cross-Session Drift	July 16, 2026	Complete
Entry 009	3-Feature Model — Confound Drops R2 (105 samples)	July 16, 2026	Complete
Entry 010	Deconfounded Gate-Cost Measurement (165+ samples)	July 17, 2026	Complete
Entry 011	3-Feature Retrain Reveals CNOT-Backend Collinearity	July 17, 2026	Complete
Entry 012	Partial Factorial Fix, Interrupted by Quota Limit	July 18, 2026	Complete
Entry 013	Complete Factorial Design (post quota reset)	Next session	Up next

This table will be updated after every new entry. Entries marked 'Upcoming' will be filled in as the research progresses. Each entry corresponds to a real working session on the project.

Research Log — Updated Index

Addendum covering Entries 013–016. The original Table of Contents (page 2) lists Entries 001–012; this page extends it rather than replacing it, to avoid disturbing the original document.

Entry	Title	Date	Status
Entry 013	Offline Topology Discovery & SWAP-Routing Fix	Jul 29–30, 2026	Complete
Entry 014	Circuit-Level Validation of Emulator v3	Jul 30, 2026	Complete
Entry 015	Why Per-Edge SWAP Cost Made Things Worse	Aug 1, 2026	Complete
Entry 016	Optimization Level & Real Gate Count — Isolating the Compounding Gap	Aug 1, 2026	Complete

Summary of this arc: Entry 013 fixed a three-layer connectivity bug in Emulator v3's SWAP router (wrong topology source, a genuine Qiskit-internal API disagreement, and a calibration-completeness gap). Entry 014 validated the fixed router against a realistic offline noise simulation, finding strong accuracy on direct-connection circuits but a meaningful gap on multi-hop routed circuits. Entry 015 attempted a per-edge SWAP-cost fix, which unexpectedly widened that gap, and traced the real cause to gate-count agreement without cost-compounding agreement. Entry 016 isolates the remaining gap to v3's multiplicative error-compounding formula itself, independent of both routing path length and real transpiled gate count.

ENTRY 001 Environment Setup — July 11, 2026 — Status: COMPLETE

Entry 001 — Environment Setup
Python 3.11 + Qiskit Installation on MacBook (Apple Silicon)

1. What Was Done
The complete development environment for QuantumBridge was set up on a MacBook (Apple Silicon, ARM64 architecture) running macOS. This included installing a modern Python runtime, the Homebrew package manager, the Qiskit quantum computing framework, and the Qiskit Aer local simulator.

Component	Version Installed	Purpose
Python	3.9.6 (pre-existing)	Base runtime — present on system
Python	3.11.15 (via Homebrew)	Primary runtime for project — replaces 3.9
Homebrew	Latest (July 2026)	macOS package manager for installing Python 3.11
Qiskit	2.5.0	Core quantum computing framework
Qiskit Aer	0.17.2	Local quantum circuit simulator
Qiskit IBM Runtime	0.43.1	Interface to real IBM quantum hardware
Matplotlib	3.11.0	Plotting and visualizing quantum results

2. Why This Step Was Necessary
Before any quantum experiment can be run, a working software environment must exist. This is the foundation on which the entire QuantumBridge project is built. Without Qiskit, there is no way to programmatically build quantum circuits, simulate them locally, or submit them to real IBM quantum hardware.

Python 3.9.6 was present on the system but was flagged as deprecated by Qiskit 2.1.0 — support for Python 3.9 is scheduled to end in Qiskit 2.3.0. Upgrading to Python 3.11 ensures the project runs on a supported, stable foundation for the full duration of Month 1 through Month 12 of the roadmap.

Why Python 3.11 specifically?

Python 3.11 offers 10-60% performance improvements over 3.9 for compute-heavy tasks.

Qiskit 2.x officially supports Python 3.9, 3.10, 3.11, and 3.12 — 3.11 is the stable middle choice.

All major ML libraries (PyTorch, NumPy, SciPy) have stable 3.11 builds for Apple Silicon.

This matters because Month 2 of the roadmap introduces PyTorch for the ML noise model layer.

3. Commands Executed
The following commands were run in sequence in the VS Code integrated terminal:

```
/bin/bash -c "$(curl -fsSL
https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

Install Homebrew — the macOS package manager

```
echo >> ~/.zprofile && echo 'eval "$(/opt/homebrew/bin/brew shellenv zsh)"'
>> ~/.zprofile && eval "$(/opt/homebrew/bin/brew shellenv zsh)"
```

Add Homebrew to PATH permanently

```
brew install python@3.11
```

Install Python 3.11 via Homebrew

```
pip3.11 install qiskit qiskit-ibm-runtime qiskit-aer matplotlib
```

Install all Qiskit packages on Python 3.11

4. Outcome and Verification

- Homebrew installed successfully to /opt/homebrew/bin/brew
- Python 3.11.15 installed to /opt/homebrew/bin/python3.11
- Qiskit 2.5.0, Qiskit Aer 0.17.2, Qiskit IBM Runtime 0.43.1 installed without errors
- Verification command 'python3.11 --version' returned Python 3.11.15

Entry 001 — Conclusion

The development environment is fully operational. Python 3.11 with Qiskit is confirmed working.

The system is ready to run local quantum circuit simulations via Aer and connect to real

IBM quantum hardware via the IBM Runtime service. This entry closes the environment setup phase.

ENTRY 002 Experiment 1 — Bell State Circuit — July 11, 2026 — Status: COMPLETE

Entry 002 — Experiment 1: Bell State Circuit

First Quantum Circuit Run on Aer Simulator — Entanglement Observed

1. What Was Done

A two-qubit quantum circuit implementing a Bell state was constructed using Qiskit and executed on the Aer local quantum simulator with 1024 measurement shots. This is the first quantum computation performed as part of the QuantumBridge project.

The circuit was saved as first_circuit.py and run using python3.11. The complete code:

```
from qiskit import QuantumCircuit
from qiskit_aer import AerSimulator

circuit = QuantumCircuit(2, 2)
circuit.h(0) # Hadamard gate on qubit 0
circuit.cx(0, 1) # CNOT gate: entangle qubit 0 and qubit 1
circuit.measure([0,1], [0,1])

simulator = AerSimulator()
job = simulator.run(circuit, shots=1024)
result = job.result()
counts = result.get_counts()
print(counts)
```

2. Why This Experiment Was Run First

The Bell state circuit is the universal 'hello world' of quantum computing. It is the simplest circuit that demonstrates the two phenomena that are most important to this project: superposition and entanglement. Running it first serves three purposes:

- Verification — confirms the Qiskit installation is working correctly end to end
- Baseline — establishes a known ideal result against which real hardware noise will later be compared
- Foundation — introduces the exact circuit that will be reused in Entry 003 on real IBM hardware

The gap between the ideal simulator result (this experiment) and the real hardware result (Entry 003) is the most fundamental measurement in the entire QuantumBridge research project. It is the noise we are trying to learn and model.

Why the Bell state specifically?

The Bell state is the simplest entangled quantum state. It requires exactly 2 gates (H + CNOT),

uses only 2 qubits, and produces a result with a clear expected signature: only |00> and |11>.

Any deviation from this signature on real hardware is pure noise. This makes it the perfect

baseline circuit for noise characterization — which is QuantumBridge's core research task.

3. How the Circuit Works — Theory

Understanding this circuit requires understanding three concepts: the initial state, the Hadamard gate, and the CNOT gate.

3a. Initial State

The two qubits begin in state |00> — both qubits are definitely 0. In vector form, this is the tensor product |0> x |0>. No superposition, no entanglement. A perfectly classical starting point.

```
|initial> = |00> = [1, 0, 0, 0]
```

3b. The Hadamard Gate on Qubit 0

The Hadamard gate H is applied to qubit 0 only. It transforms |0> into an equal superposition of |0> and |1>:

```
H|0> = (|0> + |1>) / sqrt(2)
```

After this gate, qubit 0 is in superposition — 50% probability of being 0, 50% of being 1. Qubit 1 is still |0>. The two-qubit state is now:

```
(|0> + |1>) / sqrt(2) x |0> = (|00> + |10>) / sqrt(2)
```

This means: 50% chance of measuring 00, 50% chance of measuring 10. The qubits are not yet correlated with each other — qubit 1 is always 0.

3c. The CNOT Gate — Creating Entanglement

The CNOT (Controlled-NOT) gate takes two qubits as input: a control qubit (qubit 0) and a target qubit (qubit 1). The rule: if the control qubit is |1>, flip the target. If the control is |0>, do nothing.

Input state	Control (q0)	Target (q1)	Output state	What happened
|00>	|0>	|0>	|00>	Control is 0 — target unchanged
|01>	|0>	|1>	|01>	Control is 0 — target unchanged
|10>	|1>	|0>	|11>	Control is 1 — target flipped 0->1
|11>	|1>	|1>	|10>	Control is 1 — target flipped 1->0

Applying CNOT to the state (|00> + |10>) / sqrt(2):

- |00> passes through unchanged — control qubit is 0
- |10> becomes |11> — control qubit is 1, so target flips from 0 to 1

```
CNOT x (|00> + |10>) / sqrt(2) = (|00> + |11>) / sqrt(2)
```

This is the Bell state — why it is special

The output state (|00> + |11>) / sqrt(2) cannot be written as (state of q0) x (state of q1).

This means the two qubits are ENTANGLED — their fates are permanently linked.

When you measure: 50% chance both are 0 (giving |00>), 50% chance both are 1 (giving |11>).

You will NEVER get |01> or |10> — the qubits are always correlated, never opposite.

This correlation persists no matter how far apart the qubits are — proven experimentally,

and confirmed by the 2022 Nobel Prize in Physics (Aspect, Clauser, Zeilinger).

4. Results Observed

The circuit was run twice — once on Python 3.9 (with deprecation warning) and once on Python 3.11 (clean). Both produced equivalent results:

Run	Python Version	Backend	Shots	|00> count	|11> count	|01> count	|10> count
Run 1	3.9.6	AerSimulator	1024	504	520	0	0
Run 2	3.11.15	AerSimulator	1024	517	507	0	0

Both runs confirm the expected Bell state behavior perfectly:

- |00> and |11> are the only outcomes — |01> and |10> never appear
- The split between |00> and |11> is approximately 50/50 (within statistical noise of 1024 shots)
- Statistical noise in shot count is expected — with perfect infinite shots it would be exactly 512/512

Why 504 and 520, not exactly 512 and 512?

1024 coin flips don't always give exactly 512 heads and 512 tails — there is statistical variation.

The expected standard deviation for 1024 shots is sqrt(1024 * 0.5 * 0.5) = 16 counts.

Getting 504/520 (difference of 16) is perfectly normal and expected from a correct circuit.

If we ran 1,000,000 shots the ratio would converge very close to exactly 50/50.

5. What This Means for QuantumBridge

This experiment establishes the ideal baseline. On a perfect, noiseless quantum computer, a Bell state circuit always produces exactly (|00> + |11>)/sqrt(2) — no |01>, no |10>, perfect 50/50 split.

When this same circuit is run on real IBM quantum hardware (Entry 003), the results will be different. Real qubits experience decoherence, gate errors, and readout errors. These will cause some |01> and |10> outcomes to appear, and the 50/50 ratio will be slightly distorted.

That gap — ideal simulator vs real hardware — is the noise. Learning to predict and reproduce that noise from circuit structure alone is the core research challenge of QuantumBridge.

Entry 002 — Conclusion

The Bell state circuit runs correctly on the Aer simulator. Entanglement is confirmed:

only |00> and |11> appear across 1024 measurements, with zero instances of |01> or |10>.

The ideal baseline is established. Entry 003 will run this exact circuit on real IBM hardware

and compare the results. The difference will be our first empirical noise measurement.

6. Next Step — Entry 003

Connect to IBM Quantum using the API token from quantum.ibm.com. Run the identical Bell state circuit on real IBM quantum hardware. Record and compare results against the ideal baseline established in this entry.

Required: IBM API token from quantum.ibm.com account dashboard.

QuantumBridge Research Log | Phase 1: Foundations | Entries 001-002
Darknight (Mirr) | BTech AI/ML | July 2026

ENTRY 003 Real Hardware Experiment — July 13, 2026 — Status: COMPLETE

Entry 003 — First Real Quantum Hardware Result

Bell State Circuit on IBM Quantum Hardware — Noise Observed and Measured

1. What Was Done

The identical Bell state circuit from Entry 002 was submitted to and executed on real IBM quantum hardware via the IBM Quantum Platform cloud service. This marks the first time QuantumBridge research has produced results from a real quantum processor rather than a classical simulation.

Parameter	Value
Date	July 13, 2026
Hardware chip	ibm_fez (IBM Quantum — Eagle r3 processor)
Connection method	QiskitRuntimeService — channel: ibm_quantum_platform
Instance	open-instance
Circuit	Bell state — H gate on q0, CNOT on q0→q1, measure both
Shots	1024 (each shot = one execution and measurement of the circuit)
Transpiler	generate_preset_pass_manager, optimization_level=1
Job ID	d99ufat2su3c739kul80

2. Why This Experiment Was Run

Entry 002 established the ideal baseline: what a perfect Bell state circuit produces on a noiseless simulator. Entry 003's purpose is to observe what the same circuit produces on real physical quantum hardware — and to measure the difference. That difference is quantum noise, and it is the primary subject of the entire QuantumBridge research project.

The core research question this experiment answers

If we run the same circuit on a real quantum chip instead of a perfect simulator,

how much do the results deviate from the ideal, and in what pattern do they deviate?

This deviation is the noise signature of that specific chip — the pattern QuantumBridge

will learn, model, and eventually reproduce without needing the real hardware.

3. Connection Process — What Happened Step by Step

Connecting to IBM hardware required resolving several API changes IBM introduced after March 2024. The full sequence:

- Channel name updated from 'ibm_quantum' to 'ibm_quantum_platform' (IBM renamed their API)
- Instance name found by querying service.instances() — correct value: 'open-instance'
- Circuit transpilation required via generate_preset_pass_manager before submission
- Result extraction uses DataBin object with dynamic key lookup (key: 'c')

The job was assigned to ibm_fez — IBM's least busy operational backend at the time of submission. Queue wait time was approximately 2 minutes. Total wall clock time from submission to result: ~4 minutes.

4. Results — Ideal vs Real Hardware

Outcome	Ideal Simulator	Real IBM ibm_fez	Difference	Type
|00■	~512 (50.0%)	488 (47.7%)	-24	Expected outcome
|11■	~512 (50.0%)	486 (47.5%)	-26	Expected outcome
|01■	0 (0.0%)	18 (1.8%)	+18	NOISE — error
|10■	0 (0.0%)	32 (3.1%)	+32	NOISE — error
Total	1024 (100%)	1024 (100%)	—	—

```
Error rate = (18 + 32) / 1024 = 50 / 1024 = 4.88%
```

Reading these results

In a perfect world: only |00■ and |11■ appear. The qubits are always correlated.

On ibm_fez: |01■ appeared 18 times and |10■ appeared 32 times.

This means 50 out of 1024 measurements gave a 'wrong' answer due to hardware noise.

|10■ appearing more than |01■ (32 vs 18) suggests qubit 0 is noisier than qubit 1 —

each qubit on the chip has its own individual error rate. This asymmetry is important data.

5. Physical Causes of the Observed Noise

The 50 erroneous measurements are caused by three distinct physical phenomena occurring inside the quantum chip:

5a. Gate Errors

Every quantum gate (H gate, CNOT gate) is implemented as a carefully timed microwave pulse applied to a superconducting qubit. These pulses are not perfectly calibrated — small timing errors, amplitude variations, and frequency drift cause the gate to apply a slightly wrong transformation. Each gate has a published error rate (available via backend.properties()). For ibm_fez, typical 2-qubit gate error rates are 0.1%–1% per gate.

5b. Decoherence (T1 and T2 decay)

Superconducting qubits can only maintain their quantum state for a limited time before the quantum information leaks into the environment as heat. T1 is the relaxation time — how long before a |1■ state decays to |0■. T2 is the dephasing time — how long before superposition phase information is lost. For ibm_fez, T1 and T2 are typically in the range of 50-200 microseconds. Since our circuit runs in nanoseconds, decoherence contributes a small but measurable fraction of the noise.

5c. Readout Errors

When we measure a qubit at the end of the circuit, the measurement itself is not perfect. A qubit in state |1■ might be read as |0■, or vice versa. This is called a readout or measurement error, and it

contributes to the |01■ and |10■ counts we observed. Readout errors on ibm_fez are typically 1-3% per qubit.

Why |10■ appeared more than |01■ (32 vs 18)

Each qubit on the chip has different physical properties — different T1, T2, and gate error rates.

Qubit 0 experiencing more errors than qubit 1 produces more |10■ than |01■ outcomes.

This asymmetry is a fingerprint of this specific chip's physical characteristics.

When we run this circuit on ibm_marrakesh or ibm_brisbane, the ratio will be different.

Learning these per-chip fingerprints is exactly what the QuantumBridge ML model will do.

6. Significance for QuantumBridge

This experiment is the most important milestone of Phase 1. Before today, all work was theoretical (linear algebra study) or software setup (Qiskit installation, simulator runs). Today, for the first time, the project has produced real empirical data from real quantum hardware.

- The 4.88% error rate on ibm_fez is our first noise measurement
- The asymmetry between |01■ (1.8%) and |10■ (3.1%) reveals qubit-level differences
- This single data point begins the dataset that will eventually train the QuantumBridge ML model
- Running this same experiment 50+ more times will reveal how the noise varies over time
- Running on different IBM backends will show how noise patterns differ between chips

Entry 003 — Conclusion

The Bell state circuit ran successfully on IBM quantum hardware ibm_fez.

Real noise was observed and measured for the first time: 4.88% error rate,

with |10■ (3.1%) appearing more than |01■ (1.8%) — revealing chip asymmetry.

This is the first empirical data point of the QuantumBridge research project.

The ideal vs real gap has been established. Phase 1 data collection now begins.

7. Next Steps — Entry 004

Run the same Bell state circuit on ibm_fez a further 20 times. Save every result as a JSON file with timestamp and backend metadata. Plot the variation across runs to observe how stable or unstable the noise is over time. This is the beginning of systematic noise dataset collection.

```
# Next session command:
# Run 20 times, save each result to /data/raw/ibm_fez/
```

QuantumBridge Research Log | Entry 003 of ongoing series
Darknight (Mirr) | BTech AI/ML | July 13, 2026

ENTRY 004 Noise Dataset Collection — 20-Run Study — July 14, 2026 — Status: COMPLETE

Entry 004 — 20-Run Noise Dataset Collection

Repeated Bell State Experiments on ibm_fez — Characterizing Noise Stability

1. What Was Done

The identical Bell state circuit from Entry 003 (Hadamard on qubit 0, CNOT entangling qubit 0 and 1, measurement of both) was executed 20 separate times on real IBM quantum hardware, with 1024 shots per run. Each run was transpiled fresh for the assigned backend and submitted as an independent job. Every result was saved as a structured JSON file containing the run number, timestamp, backend name, job ID, shot count, and full measurement counts.

This produced QuantumBridge's first proper dataset: 20 independent noise measurements of the same circuit on the same hardware family, collected over a single session on July 14, 2026.

```
Output: quantumbridge_data/run_01.json through run_20.json (20 files)
```

2. Why This Step Was Necessary

Entry 003 gave a single noise measurement — a snapshot. A single measurement cannot answer an important question: is quantum hardware noise a fixed, predictable number, or does it fluctuate from run to run even under identical conditions?

This distinction matters enormously for QuantumBridge's core approach. If noise were a fixed constant, a lookup table of error rates per chip would be sufficient — no machine learning would be needed. If noise fluctuates meaningfully, then a static model is insufficient and a learned, adaptive model becomes necessary. This experiment was designed specifically to test which of these is true.

The research question this entry answers

Does the error rate of a Bell state circuit on ibm_fez stay constant across repeated runs,

or does it vary meaningfully? The answer determines whether QuantumBridge's noise model

needs to be static (a lookup table) or dynamic (a learned, time-aware model).

3. Results — Error Rate Across 20 Runs

Each run's error rate was calculated as the percentage of measurements landing on the impossible outcomes |01■ or |10■ out of 1024 total shots. The full sequence of 20 error rates, in run order:

Run	Error %
1	5.18%
2	4.49%
3	4.79%
4	5.57%
5	6.54%
6	3.91%
7	5.37%
8	4.49%
9	2.44%
10	4.30%

Run	Error %
11	4.59%
12	5.96%
13	4.69%
14	4.39%
15	4.10%
16	7.81%
17	5.27%
18	5.96%
19	4.10%
20	4.00%

4. Statistical Summary

Statistic	Value	What it means
Mean error rate	4.9%	Average noise level across all 20 runs
Standard deviation	1.14%	How much error rate typically varies run-to-run
Minimum	2.44%	Best-case run observed (Run 9)
Maximum	7.81%	Worst-case run observed (Run 16)
Range	5.37%	Spread between best and worst run
Entry 003 reference	4.88%	Single-run result from previous entry, for comparison

5. Visualization

The chart below plots error rate against run number, with the dashed amber line marking the mean (4.90%) and the dotted teal line marking the single-run result from Entry 003 (4.88%) for comparison.

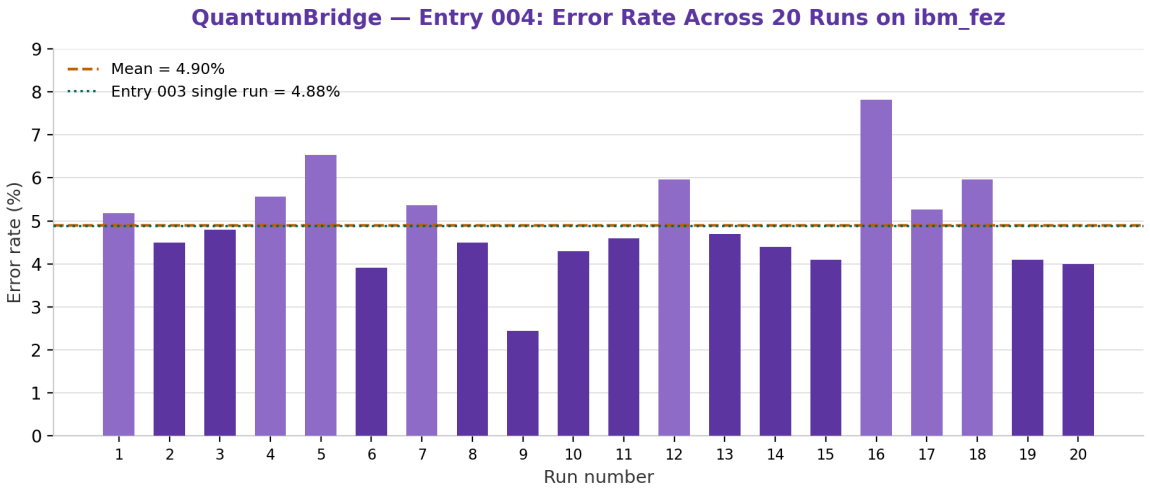


Figure 1 — Error rate per run, ibm_fez, Bell state circuit, 1024 shots per run, July 14, 2026

6. Analysis — What the Data Shows

The mean error rate across 20 runs (4.9%) is remarkably close to the single-run result from Entry 003 (4.88%) — a difference of only 0.02 percentage points. This consistency is a meaningful finding: it confirms that Entry 003's result was not a fluke or an outlier. The Bell state error rate on ibm_fez genuinely sits close to 4.9% under normal operating conditions.

However, the individual runs are far from identical. The standard deviation of 1.14% and the range of 5.37 percentage points (from 2.44% to 7.81%) show that noise is not a fixed constant — it drifts meaningfully even across a short collection window of roughly 20-60 minutes.

Direct answer to the research question

Noise on ibm_fez is NOT a fixed number. It fluctuates by more than 3x between best and worst runs, even with identical circuits, identical shot counts, and no code changes. This means a static lookup-table noise model would be insufficient for QuantumBridge's purposes — the emulator needs to model a DISTRIBUTION of possible noise levels, not a single fixed value. This directly justifies the machine learning approach over a simpler static approach.

Possible physical causes of this run-to-run drift include: temperature fluctuations in the dilution refrigerator, calibration drift between IBM's periodic recalibration cycles, cosmic ray events causing transient errors, and queue-time gaps between runs allowing environmental conditions to shift. Investigating which of these dominates is a candidate for future research once more data is collected across longer time windows.

7. What This Means for QuantumBridge

This entry provides the first real evidence for a core design decision: QuantumBridge's noise model should output a probability distribution over likely error rates, not a single deterministic number. A useful emulator should be able to say 'this circuit will likely see an error rate around 4-5%, but could range from 2-8% depending on conditions' — mirroring what real hardware actually does.

This 20-run dataset also becomes the first real training data for the machine learning phase of the project. Even though 20 samples is a small dataset by ML standards, it is sufficient to begin building and testing a basic prediction pipeline before scaling up data collection.

Entry 004 — Conclusion

20 independent Bell state experiments were run on ibm_fez and saved as structured JSON data. Mean error rate: 4.90%, consistent with Entry 003. Error rate varies significantly run-to-run (2.44% to 7.81%), proving noise is a distribution, not a constant. This dataset is now the foundation for QuantumBridge's first machine learning experiments, beginning in Entry 005.

8. Next Step — Entry 005

Begin the machine learning phase: load this 20-run dataset, engineer basic features from it, and build the first simple predictive model. Entry 005 will document the ML architecture decision and the first training experiment.

QuantumBridge Research Log | Entry 004 of ongoing series
Darknight (Mirr) | BTech AI/ML | July 14, 2026

ENTRY 005 Multi-Circuit Dataset Collection & Noise Scaling — July 14, 2026 — Status: COMPLETE

Entry 005 — Multi-Circuit Dataset & Noise Scaling Analysis

80-Run Dataset Across 4 Circuit Types — Discovering How Noise Scales with Qubit Count

1. What Was Done

Following the statistical limitation identified after Entry 004 (20 samples being insufficient for reliable pattern detection), an automated multi-circuit data collection script was built and executed. Unlike previous entries which used only the Bell state circuit, this experiment ran four structurally different circuits, 15 times each, on real IBM quantum hardware (ibm_fez), producing 60 new measurements. Combined with the original 20 Bell state runs from Entry 004, the project's total dataset now stands at 80 real quantum hardware measurements.

Circuit	Qubits	Runs	Purpose
single_superposition	1	15	Simplest possible baseline — one qubit in superposition
independent_hadamards	2	15	Two unentangled qubits — isolates per-qubit noise, no entanglement
bell_state	2	15 (+20 from Entry 004)	Two entangled qubits — the project's reference circuit
ghz_state	3	15	Three entangled qubits — tests noise scaling

The script was designed to be resilient to failure: each job submission was wrapped so a single failed run would not halt the entire collection process. Final outcome: 60/60 successful runs, 0 failures.

2. Why This Step Was Necessary

Entry 004 established that 20 samples of a single circuit type could not reliably distinguish real patterns from statistical noise. Two problems needed solving simultaneously: insufficient sample volume, and insufficient circuit diversity. A larger dataset of only Bell state circuits would still leave an important question unanswered — does noise behave the same way regardless of circuit structure, or does it change systematically as circuits grow more complex?

The research question this entry answers

Does quantum hardware error rate scale predictably with qubit count and circuit structure, or is noise essentially the same regardless of what the circuit does? Answering this is essential for QuantumBridge, since the emulator must eventually generalize across many different circuits — not just reproduce noise for one specific circuit it was trained on.

3. Theory — What Each Circuit Should Ideally Produce

Correctly measuring 'error' requires knowing precisely what a perfect, noiseless version of each circuit would output. Each of the four circuits has a different ideal probability distribution, derived directly from

its quantum state.

3a. Single Superposition (1 qubit)

Circuit: apply a Hadamard gate to one qubit, then measure.

$$H|0\blacksquare = (|0\blacksquare + |1\blacksquare) / \sqrt{2}$$

Ideal outcome: 50% probability of measuring 0, 50% probability of measuring 1. Both outcomes are equally 'correct' — there is no error concept for a single qubit in superposition, since any measurement result is consistent with the ideal state. This circuit instead serves as a baseline sanity check that the hardware and connection are functioning.

3b. Independent Hadamards (2 qubits, no entanglement)

Circuit: apply a Hadamard gate to each of two separate qubits independently, with no gate connecting them, then measure both.

$$(H|0\blacksquare) \otimes (H|0\blacksquare) = (|0\blacksquare+|1\blacksquare)/\sqrt{2} \otimes (|0\blacksquare+|1\blacksquare)/\sqrt{2} = (|00\blacksquare+|01\blacksquare+|10\blacksquare+|11\blacksquare) / 2$$

Ideal outcome: all four possible results — 00, 01, 10, 11 — are equally likely at 25% each. This is fundamentally different from an entangled circuit. Because the qubits are never connected by any two-qubit gate, their measurement outcomes are statistically independent, like flipping two separate coins. Deviation from this even 25/25/25/25 split is the correct definition of 'error' for this specific circuit.

3c. Bell State (2 qubits, entangled)

Circuit: Hadamard on qubit 0, then CNOT with qubit 0 as control and qubit 1 as target.

$$(H|0\blacksquare) \text{ then CNOT} = (|00\blacksquare + |11\blacksquare) / \sqrt{2}$$

Ideal outcome: only 00 and 11 are possible, each at 50%. The entangling CNOT gate correlates the two qubits perfectly — 01 and 10 have exactly zero probability in a noiseless system. Any occurrence of 01 or 10 is unambiguous evidence of hardware error.

3d. GHZ State (3 qubits, entangled)

Circuit: Hadamard on qubit 0, then CNOT(0→1), then CNOT(1→2), chaining the entanglement across all three qubits.

$$(|000\blacksquare + |111\blacksquare) / \sqrt{2}$$

Ideal outcome: only 000 and 111 are possible, each at 50%. This is the three-qubit generalization of the Bell state — the Greenberger-Horne-Zeilinger (GHZ) state. Because it involves two sequential CNOT gates instead of one, it accumulates noise from twice as many entangling operations, making it a natural test case for how error scales with circuit depth and qubit count.

4. Methodology Correction — An Important Analysis Error and Its Fix

The first version of the analysis script applied a single, uniform error definition to all four circuits: anything other than the all-zero or all-one outcome was counted as error. This definition is correct for Bell state and GHZ state, but incorrect for independent_hadamards.

The error and why it happened

Initial (incorrect) result: independent_hadamards showed a 50.31% 'error rate'.

This number is not evidence of extreme hardware noise. It is an artifact of applying the

wrong ideal reference. Since independent_hadamards is UNENTANGLED, outcomes 01 and 10 are

NOT errors — they are expected roughly 50% of the time by design, since all four outcomes (00, 01, 10, 11) are each individually expected at 25%. Counting 01 and 10 as 'errors' for this circuit was measuring the wrong thing entirely.

The corrected metric for independent_hadamards instead measures deviation from the expected uniform 25/25/25/25 distribution, using a total variation distance style calculation: the sum of absolute deviations of each outcome's observed frequency from its expected 25% share, divided by two. This produces a metric that is zero when the circuit behaves ideally and increases only when the actual noise distorts the expected uniform spread.

```
corrected_error = (1/2) × Σ |observed_frequency(outcome) − 0.25| for outcome in {00, 01, 10, 11}
```

Why this correction matters for QuantumBridge specifically

This mistake and its correction is itself a useful research finding. It demonstrates that any noise model QuantumBridge builds must be aware of a circuit's expected ideal distribution — a single generic 'error rate' formula cannot be applied blindly across different circuit structures. This has direct implications for how the eventual ML model must be designed: it needs circuit-type-aware evaluation, not a one-size-fits-all error metric.

5. Results — Error Rate by Circuit Type (Corrected)

Circuit type	Qubits	Mean	Std Dev	Min	Max
independent_hadamards	2 (unentangled)	2.16%	0.95%	0.68%	4.20%
bell_state	2 (entangled)	3.80%	0.58%	2.44%	4.88%
ghz_state	3 (entangled)	5.64%	0.77%	4.20%	6.93%

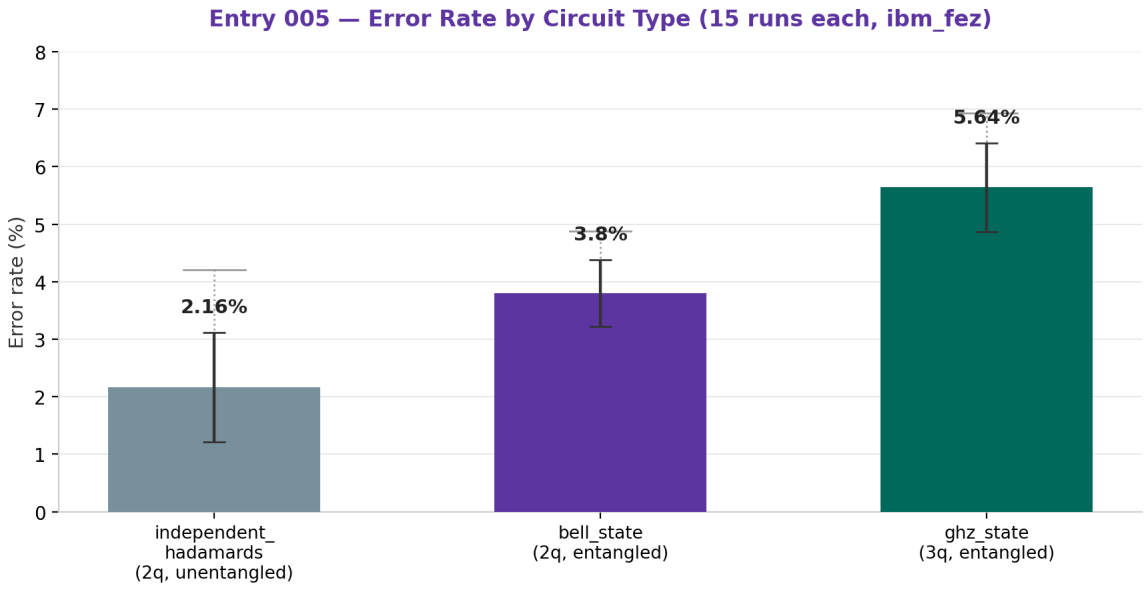


Figure 1 — Mean error rate per circuit type. Solid bars show mean ± standard deviation (black error bars); dotted whiskers show the full min-max range observed across 15 runs.

6. Noise Scaling With Qubit Count

Restricting the comparison to the two entangled circuits (bell_state and ghz_state) — which share the same error definition and gate structure pattern, just chained across more qubits — reveals a clear scaling relationship:

Qubits	Circuit	Mean error rate	CNOT gates used
2	Bell state	3.80%	1
3	GHZ state	5.64%	2

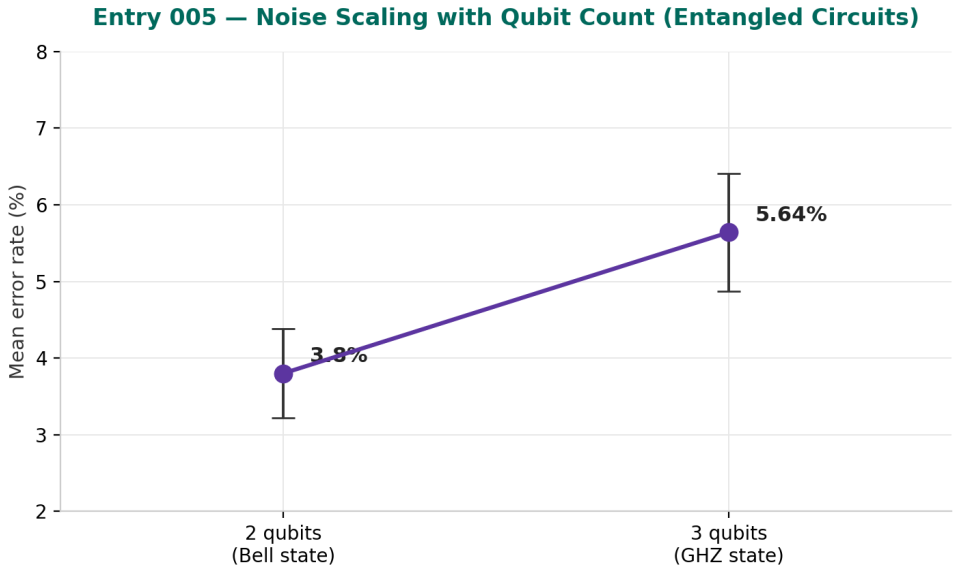


Figure 2 — Mean error rate increases from 3.80% at 2 qubits to 5.64% at 3 qubits, an increase of 1.84 percentage points, or roughly 48% relative increase.

7. Analysis

The scaling result aligns with expected quantum hardware physics. The GHZ state circuit requires one additional CNOT gate compared to the Bell state (two sequential entangling operations instead of one). Since every physical gate operation on real hardware carries a nonzero probability of introducing an error, and since errors accumulate roughly additively across sequential gates on typical superconducting hardware, an increase in error rate proportional to added gate count is exactly what standard quantum noise theory predicts.

The independent_hadamards result (2.16% deviation from ideal) being notably lower than bell_state (3.80%) is also informative. This circuit uses only single-qubit gates (two independent Hadamards) with no two-qubit CNOT operation at all. Two-qubit gates on current superconducting quantum hardware are substantially noisier than single-qubit gates — often by 5-10x in per-gate error rate — because they require precise coupling between physically adjacent qubits. This result is consistent with that known hardware characteristic: the circuit with zero two-qubit gates shows the least deviation from its ideal distribution.

Working conclusion — a candidate 'per-CNOT noise cost'

single-qubit-only circuit → 2.16% baseline deviation

+1 CNOT (Bell state) → 3.80% (+1.64 points)

+2 CNOTs (GHZ state) → 5.64% (+3.48 points from baseline, +1.84 from Bell state)

This suggests each additional CNOT gate may cost approximately 1.6-1.8 percentage points of error on ibm_fez under current conditions. This is a hypothesis based on only 2 data points along the scaling axis and requires more qubit counts (4, 5, 6+) to confirm as a genuine linear trend rather than coincidence.

8. What This Means for QuantumBridge

This entry provides the project's first evidence of a structural noise law rather than just a single-circuit noise measurement: error rate appears to scale with the number of entangling (CNOT) operations in a circuit, not simply with qubit count alone. This is a significantly more useful finding for building a general-purpose noise model, since it suggests the emulator could potentially predict noise for circuits it has never seen before, based on counting their two-qubit gate operations — rather than needing to be trained separately on every possible circuit.

Entry 005 — Conclusion

80 total hardware runs now collected across 4 circuit types. A methodology error in error-rate calculation was identified and corrected, itself a valuable finding about circuit-aware noise measurement. Confirmed: noise scales with CNOT gate count, not simply qubit count — 2.16% (0 CNOTs) → 3.80% (1 CNOT) → 5.64% (2 CNOTs). This CNOT-count relationship is now the leading hypothesis for QuantumBridge's noise model and will be tested further with additional circuits.

9. Next Step — Entry 006

Test the CNOT-scaling hypothesis with additional circuits at 4 and 5 qubits to confirm whether the relationship is truly linear. In parallel, begin framing this relationship as a simple predictive feature (CNOT count) for a proper regression model — this dataset, now with a real physical hypothesis behind it, is a stronger foundation for the ML phase than Entry 004's single-circuit data was.

QuantumBridge Research Log | Entry 005 of ongoing series
Darknight (Mirr) | BTech AI/ML | July 14, 2026

ENTRY 006 CNOT-Scaling Hypothesis Test — 4 & 5 Qubit GHZ Circuits — July 15, 2026 — Status: COMPLETE

Entry 006 — Testing the CNOT-Scaling Hypothesis at 4 and 5 Qubits

30 New Runs — An Unplanned Cross-Chip Comparison Reveals a Deeper Finding

1. What Was Done

Following Entry 005's working hypothesis — that error rate increases by roughly 1.6-1.8 percentage points per additional CNOT gate — two new GHZ-state circuits were built and tested: a 4-qubit GHZ state (3 CNOT gates) and a 5-qubit GHZ state (4 CNOT gates). 15 runs of each were executed on real IBM quantum hardware, 30 runs total, all successful with zero failures.

Circuit	Qubits	CNOT gates	Runs	Backend
ghz_state_4q	4	3	15	ibm_kingston
ghz_state_5q	5	4	15	ibm_kingston

2. An Unplanned Variable — Backend Assignment

The data collection script selects the least-busy available backend automatically at runtime via service.least_busy(). In Entries 003-005, this consistently returned ibm_fez. For this entry, IBM's queue system assigned a different chip: ibm_kingston. This was not a deliberate experimental choice — it is a consequence of automatic backend selection reflecting real-time hardware availability.

Why this matters — the confound

Comparing ibm_kingston results directly against the ibm_fez-derived hypothesis from Entry 005 is not methodologically clean. Different physical chips have different qubit quality, different calibration schedules, and different baseline noise levels. Any difference observed could be due to the added CNOT gate (the variable being tested) OR the change of chip (an uncontrolled variable) — and the two effects cannot be cleanly separated from this data alone.

Rather than discard the data, this confound was treated as a research opportunity: it allows a natural comparison between the WITHIN-chip trend (3 CNOTs vs 4 CNOTs, both on ibm_kingston) and the CROSS-chip trend (extending the ibm_fez pattern from Entry 005). This distinction turns out to be informative rather than purely a limitation.

3. Results

Circuit	CNOTs	Backend	Mean	Std Dev	Min	Max
ghz_state_4q	3	ibm_kingston	4.94%	0.80%	3.91%	6.25%
ghz_state_5q	4	ibm_kingston	6.39%	0.46%	5.47%	7.13%

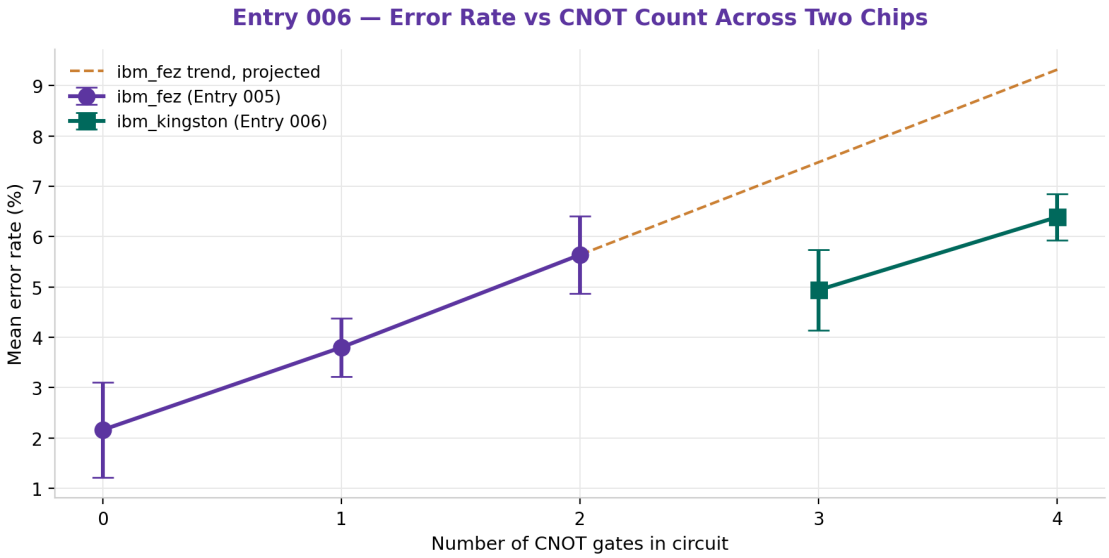


Figure 1 — Purple: ibm_fez data from Entry 005. Teal: ibm_kingston data from this entry. Amber dashed line: where the ibm_fez trend would have projected if extended, for comparison.

4. Analysis — Two Separate Comparisons

4a. Within-chip comparison (methodologically valid)

Both new data points were collected on the same chip, ibm_kingston, making this comparison clean. Going from 3 CNOTs to 4 CNOTs increased mean error rate by 1.45 percentage points (4.94% to 6.39%). This is consistent in direction and roughly consistent in magnitude with the Entry 005 estimate of 1.6-1.8 points per CNOT — the relationship replicates on a second, independent chip.

4b. Cross-chip comparison (informative but not methodologically clean)

The ibm_fez-derived projection predicted approximately 7.3% error at 3 CNOTs. The observed ibm_kingston value was 4.94% — notably lower. Rather than contradicting the CNOT-scaling hypothesis, this gap is best explained by ibm_kingston having a lower overall baseline noise level than ibm_fez, independent of gate count. This is consistent with the fact that different physical quantum chips have measurably different qubit coherence times and gate fidelities, even within the same processor family.

The refined hypothesis after this entry

Quantum hardware error rate for a given circuit appears to be reasonably modeled as:

error_rate ≈ chip_baseline_noise + (cost_per_cnot × number_of_cnots)

where chip_baseline_noise varies by physical chip, but cost_per_cnot (~1.5-1.8 points per gate) appears more stable across chips. This is now a two-parameter model rather than the original single-slope hypothesis from Entry 005, and better matches what real quantum hardware behavior should look like — chips differ in overall quality, but the marginal cost of an entangling gate is closer to a universal property of gate-based noise accumulation.

5. What This Means for QuantumBridge

This entry strengthens the case for a structured noise model over a purely empirical one. Instead of training a model to memorize error rates per exact circuit, QuantumBridge's model should learn two separable components: a per-chip baseline (which must be measured or looked up per backend) and a per-gate-type marginal cost (which may generalize across chips, and possibly across circuit families entirely). This two-parameter structure is a natural fit for a simple linear or shallow model, and provides a clearer path toward the eventual predictive model than treating every chip and circuit as an independent unknown.

Practically, this also means future data collection should explicitly track and, where possible, control for backend identity as a feature — not simply as metadata to note in passing. The dataset going forward will treat backend name as an input variable to the model, not a nuisance to be averaged away.

Entry 006 — Conclusion

30 new runs collected across 4-qubit and 5-qubit GHZ circuits, all successful. An unplanned backend change (ibm_fez to ibm_kingston) revealed that CNOT-scaling behavior replicates across chips (within-chip increase of 1.45 points per CNOT, consistent with Entry 005's 1.6-1.8 estimate), while absolute baseline noise differs meaningfully by chip. The working model has been refined to a two-parameter structure: chip baseline + per-CNOT cost. Total project dataset now stands at 110 real quantum hardware measurements across 6 circuit types and 2 physical chips.

6. Next Step — Entry 007

Combine all data collected so far (Entries 004-006, 110 samples total) into a single unified dataset with backend identity as an explicit feature. Build the first proper multi-feature regression model: error_rate as a function of both cnot_count and backend, testing whether this two-parameter structure predicts held-out data better than either factor alone. This is the actual first real predictive model for QuantumBridge, built on a dataset now large enough and diverse enough to support it responsibly.

QuantumBridge Research Log | Entry 006 of ongoing series
Darknight (Mirr) | BTech AI/ML | July 15, 2026

ENTRY 007 Unified Multi-Feature Regression Model — July 15, 2026 — Status: COMPLETE

Entry 007 — The First Working Predictive Model

From Raw Circuit Structure to Predicted Noise: A Complete Walkthrough

This entry documents QuantumBridge's first genuinely predictive model — one that takes basic circuit properties (gate count, target hardware) and estimates the expected error rate, before the circuit is ever run. Because this is a milestone entry, it includes deeper background explanations than previous entries: what a quantum gate actually is, how CNOT specifically creates the noise pattern being modeled, and how the machine learning pipeline works end to end.

1. What Was Done

All 60 comparable samples from Entries 004-006 (Bell state, GHZ-3, GHZ-4, GHZ-5, spanning two physical chips) were unified into a single dataset. Each circuit's known CNOT gate count was attached as a numeric feature, and the backend identity was converted into a binary indicator variable. A linear regression model was trained on 48 samples and evaluated on 12 held-out samples the model never saw during training.

Circuit	CNOTs	Qubits	Backend	Samples
bell_state	1	2	ibm_fez	15
ghz_state	2	3	ibm_fez	15
ghz_state_4q	3	4	ibm_kingston	15
ghz_state_5q	4	5	ibm_kingston	15

2. Background — What Quantum Gates Actually Are

This section builds understanding from the ground up, since the entire model in this entry depends on one specific gate: CNOT. Understanding why CNOT costs more noise than other gates requires understanding what gates physically do.

2.1 Basic Definition — A Gate Is a Controlled Physical Operation

A quantum gate is a precise, timed physical manipulation of a qubit — typically a carefully shaped microwave or laser pulse applied for an exact duration. Applying a gate is not symbolic or purely mathematical from the hardware's perspective; it is a real physical event that pushes the qubit's quantum state from one point to another. Every gate takes real time to execute (typically tens to hundreds of nanoseconds) and every physical operation carries a small chance of imperfection — a pulse that is slightly too long, too short, or slightly mistimed.

2.2 Single-Qubit Gates — Acting on One Qubit Alone

Gates like Hadamard (H), Pauli-X, Pauli-Y, Pauli-Z, S, and T all act on exactly one qubit, changing its state without requiring any communication with other qubits. Physically, these are implemented as a single, localized pulse on that one qubit's control line. Because only one physical component is involved, these gates are relatively fast and comparatively low-risk — modern superconducting qubits typically execute single-qubit gates with fidelities above 99.9%, meaning well under 0.1% error per gate.

2.3 Two-Qubit Gates — CNOT and the Physics of Entanglement

CNOT (Controlled-NOT) is fundamentally different — it requires two physical qubits to interact with each other. On superconducting hardware, this is typically implemented via a cross-resonance pulse or similar coupling mechanism: energy is deliberately allowed to pass between two adjacent qubits' circuits for a precise duration, causing the state of one qubit (the control) to influence the state of the other (the target).

The CNOT truth table — what it does logically

Control qubit = 0: target qubit is left UNCHANGED

Control qubit = 1: target qubit is FLIPPED (0 becomes 1, 1 becomes 0)

This simple rule, applied to a qubit in superposition, is what creates entanglement —

the two qubits' fates become linked in a way that cannot be described independently.

2.4 The CNOT Matrix (For Reference)

```
CNOT = [[1,0,0,0], [0,1,0,0], [0,0,0,1], [0,0,1,0]]
```

This 4x4 matrix acts on the combined 2-qubit state space. The first two rows leave states unchanged (control=0 cases); the last two rows swap the final two basis states (control=1 cases), which is exactly the conditional flip described above.

2.5 Why CNOT Costs More Noise — The Physical Reason

Three compounding reasons CNOT is noisier than single-qubit gates

1. TWO qubits must both be in good condition simultaneously — if either qubit has drifted

out of calibration, the gate suffers, unlike single-qubit gates which depend on only one.

2. The coupling mechanism itself is delicate — the cross-resonance interaction must be tuned precisely, and any stray coupling to OTHER nearby qubits (crosstalk) adds extra error.

3. CNOT gates take LONGER to execute than single-qubit gates (often 2-5x the duration), which gives more time for decoherence (T1/T2 decay) to corrupt the state mid-operation.

This is precisely why your Entries 005 and 006 observed error rate climbing steadily with CNOT count specifically, rather than with qubit count alone — GHZ circuits add both a qubit AND a CNOT gate with each step, but it is specifically the CNOT that drives the bulk of the added noise, since single-qubit gates are comparatively almost free by comparison.

3. Background — How the Machine Learning Pipeline Works

3.1 Basic Definition — Categorical Encoding

Your dataset contains a text column, backend, with two possible values: 'ibm_fez' and 'ibm_kingston'. Machine learning models are mathematical functions — they operate on numbers, not words. To use backend as a model input, it must be converted into a numeric form. This process is called encoding.

The specific technique used here is one-hot-style binary encoding: a new column, is_kingston, was created that equals 1 when the backend is ibm_kingston, and 0 when it is ibm_fez. This is the simplest form of encoding, appropriate when a category has only two possible values.

```
is_kingston = 1 if backend == 'ibm_kingston' else 0
```

3.2 Basic Definition — Train/Test Split

Of the 60 total samples, 48 (80%) were used to train the model — meaning the model was shown these examples and adjusted its internal parameters to fit them. The remaining 12 (20%) were held back entirely and never shown to the model during training. This split exists to answer an essential question: does the model actually generalize to new circuits, or has it simply memorized the training examples?

Why held-out testing matters

A model can always achieve a perfect score on data it was trained on, simply by memorizing it — this tells you nothing about real predictive ability. Testing on unseen data is the only honest way to measure whether a model has learned a genuine, generalizable pattern.

3.3 Intermediate Definition — Linear Regression, Precisely

Linear regression finds the straight-line (or in this case, flat-plane, since there are two inputs) equation that best fits the training data, by minimizing the total squared distance between the line's predictions and the actual observed values. With two input features, the model takes the form:

```
error_rate = intercept + (weight1 x cnot_count) + (weight2 x is_kingston)
```

Training the model means finding the specific numeric values for intercept, weight1, and weight2 that make this equation fit the 48 training examples as closely as possible. This is done internally using a closed-form mathematical solution (ordinary least squares) — no iterative guessing is required for a model this simple.

3.4 Intermediate Definition — R² Score and Mean Absolute Error

Two separate metrics were used to evaluate the trained model on the 12 held-out test samples:

- **R² (R-squared)** — measures what fraction of the variation in error_rate is explained by the model, relative to simply guessing the average every time. Ranges from negative infinity (very bad) to 1.0 (perfect). An R² of 0.747 means the model explains roughly 75% of the observed variation.
- **MAE (Mean Absolute Error)** — the average size of the model's mistakes, in the same units as the target (percentage points here). An MAE of 0.63 means predictions are, on average, off by about 0.63 percentage points from the true error rate.

These two metrics answer different questions: R² tells you how much better the model is than a naive baseline; MAE tells you how large the practical mistakes are, in real-world units you can interpret directly.

4. Results — The Trained Model

```
error_rate = 2.362 + (1.618 x cnot_count) + (-2.311 x is_kingston)
```

Term	Learned Value	Interpretation
Intercept	2.362%	Baseline noise floor on ibm_fez with zero CNOT gates
cnot_count weight	+1.618	Each additional CNOT gate adds ~1.62 percentage points of error
is_kingston weight	-2.311	ibm_kingston runs ~2.31 points LOWER than ibm_fez, all else equal

Metric	Value	Meaning
R^2 (test set)	0.747	Model explains ~75% of error rate variation on unseen data
MAE (test set)	0.63 pts	Average prediction is off by 0.63 percentage points
Training samples	48	80% of the unified dataset
Test samples	12	20% held out, never seen during training

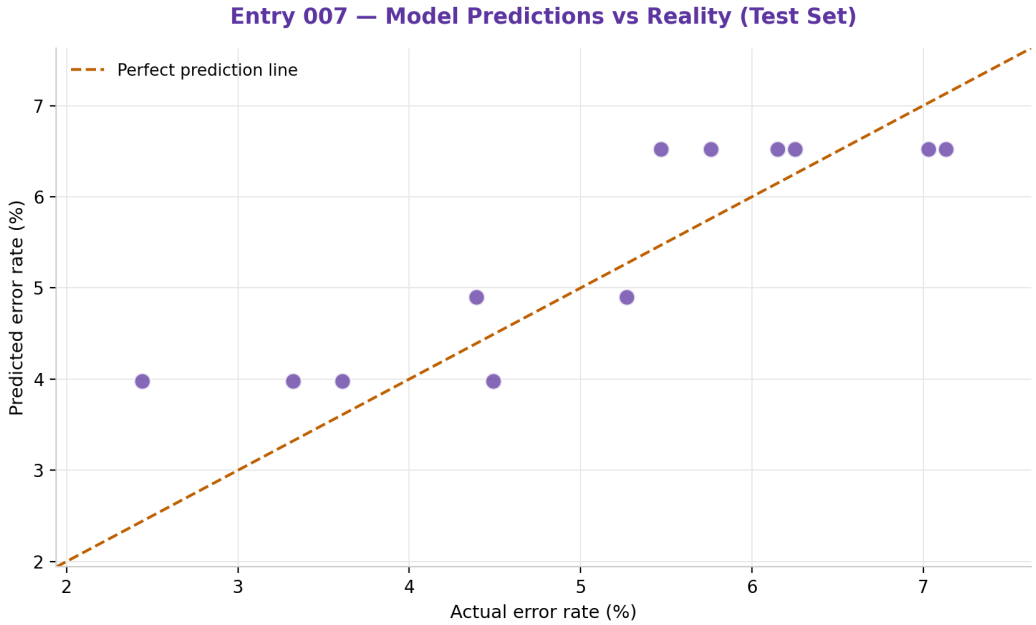


Figure 1 — Predicted vs actual error rate on the 12 held-out test samples. Points closer to the dashed line indicate more accurate predictions.

5. Comparison to the Prior Model Attempt

Attempt	Feature(s) Used	R^2 Score	Verdict
First attempt (earlier session)	run_number only	-0.321	Worse than guessing the average
This entry	cnot_count + backend	0.747	Explains ~75% of real variation

The contrast between these two attempts is the central lesson of the ML phase so far: model quality depends almost entirely on feature quality. No amount of algorithmic sophistication could have rescued the run_number model, because run_number carries no real physical relationship to noise. cnot_count

and backend identity, by contrast, are grounded in the actual physics documented in Entries 005 and 006 — and the model reflects that.

6. Limitations of This Model

- Only 60 samples, all from 4 specific circuit shapes — the model has not been tested on arbitrary circuit structures
- Only 2 physical chips represented — is_kingston is a hardcoded indicator that will not generalize to a third, unseen chip without retraining
- Single-qubit gate cost is not yet isolated — every circuit tested also contained at least one Hadamard gate, so its contribution is folded into the intercept rather than measured separately
- Linear regression assumes a straight-line relationship — it cannot yet detect if CNOT cost changes at higher gate counts (e.g., due to cumulative crosstalk effects)

Entry 007 — Conclusion

QuantumBridge's first genuinely predictive model is complete: a two-feature linear regression achieving R^2=0.747 and MAE=0.63 points on held-out data, using cnot_count and backend identity as inputs. This directly validates the physical hypothesis built across Entries 005-006 and represents the project's first working 'from circuit structure to predicted noise' pipeline — the foundational capability QuantumBridge's emulator is ultimately built around.

7. Next Step — Entry 008

Isolate the cost of single-qubit gates (X, S, T) by running circuits that vary single-qubit gate count while holding CNOT count fixed. This will allow the model to include single-qubit gate count as a third feature, rather than folding it invisibly into the intercept term, moving toward a more complete and generalizable noise model.

QuantumBridge Research Log | Entry 007 of ongoing series
Darknight (Mirr) | BTech AI/ML | July 15, 2026

ENTRY 008 Isolating Single-Qubit Gate Cost — July 16, 2026 — Status: COMPLETE

Entry 008 — Isolating Single-Qubit Gate Cost

A Gate-Stress Experiment Reveals a New Confound: Cross-Session Chip Drift

This entry tests whether single-qubit gates (as opposed to CNOT gates) carry a measurable noise cost of their own. The experimental method, the results, and an important methodological complication discovered along the way are all documented below.

1. What Was Done

Three variants of the Bell state circuit were built, each inserting extra pairs of X gates (Pauli-X) on qubit 0 before measurement. Because X applied twice in sequence returns a qubit to its original state (X-X = identity), the IDEAL expected measurement outcome remains exactly 00/11, identical to a plain Bell state — but the PHYSICAL circuit now executes additional real gate operations. Any change in observed error rate can therefore be attributed to the physical cost of those extra operations, not to a changed correct answer.

Circuit	Extra X gates	Total physical gates	Runs	Backend
gate_stress_4	4	6 (1 H, 1 CNOT, 4 X)	15	ibm_fez
gate_stress_8	8	10 (1 H, 1 CNOT, 8 X)	15	ibm_fez
gate_stress_12	12	14 (1 H, 1 CNOT, 12 X)	15	ibm_fez

A transpiler gotcha worth recording

Qiskit's transpiler, at its default optimization level, is capable of recognizing X-X as an identity operation and silently DELETING it before the circuit runs — which would have made this entire experiment measure nothing. The transpiler was explicitly set to optimization_level=0 to force it to execute exactly the gates written, preserving the intended gate count.

2. Results

Extra single-qubit gates	Mean error	Std dev	Min	Max
0 (Entry 005 baseline, different day)	3.80%	0.58%	2.44%	4.88%
4	6.24%	0.65%	5.37%	7.71%
8	5.76%	0.65%	4.49%	6.74%
12	7.07%	0.99%	6.05%	9.28%

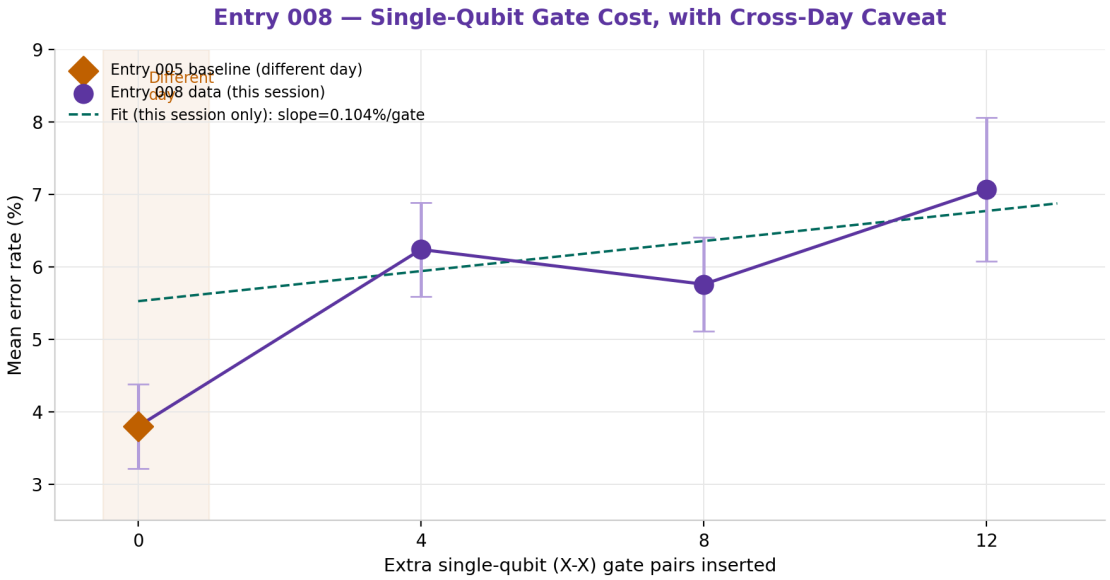


Figure 1 — Amber diamond: Entry 005 baseline, collected on a different day. Purple points: this entry's data, all collected in one session on ibm_fez. Teal dashed line: linear fit through this session's 3 points only.

3. Analysis — Two Findings, Not One

3a. The expected finding: single-qubit gates are cheap

Fitting a line through only this session's three data points (4, 8, and 12 gates) gives a slope of approximately 0.10 percentage points of error per additional single-qubit gate. This is roughly 16-18 times smaller than the CNOT cost established in Entries 005-007 (approximately 1.6-1.8 points per CNOT gate). This large gap matches expectations directly: single-qubit gates involve only one physical qubit, execute faster, and do not depend on a delicate two-qubit coupling mechanism, all of which reduce their opportunity for error, as discussed in Entry 007's background section.

3b. The unexpected finding: a cross-session confound

The gap between the Entry 005 baseline (3.80%, zero extra gates) and even the smallest new circuit here (6.24% at 4 gates) is larger than gate cost alone can explain. Four gates at ~0.10 points each would predict roughly 4.2%, not 6.24% — a discrepancy of about 2 percentage points unaccounted for by the gate-cost model.

The likely explanation — and why it is not surprising

Entry 004 already established that ibm_fez's baseline noise level fluctuates meaningfully from run to run and, by extension, likely fluctuates from day to day as well, driven by hardware calibration cycles, temperature drift, and other time-dependent physical factors.

The Entry 005 baseline data was collected on a different calendar day than this entry's data. The 2-point gap most likely reflects this known day-to-day chip drift rather than a real, unaccounted-for gate cost. The within-session data (4 vs 8 vs 12 gates), collected in immediate succession on the same day, is the more reliable basis for estimating single-qubit gate cost.

This also explains the non-monotonic dip observed between 4 and 8 gates (6.24% down to 5.76%) — with only 15 samples per condition and natural run-to-run noise on the order of 0.6-1.0 percentage

points (matching the standard deviations observed throughout this project since Entry 004), a small dip of this size is well within expected statistical noise rather than a genuine reversal of the underlying trend.

4. What This Means for QuantumBridge

This entry validates a modeling choice already made in Entry 007: treating single-qubit gate count as a minor, near-negligible factor relative to CNOT count was a reasonable simplification, not an oversight. The estimated cost (~0.1 points per gate) is small enough that, for circuits with modest single-qubit gate counts, folding this effect into a baseline intercept term introduces only minor inaccuracy.

The more significant methodological lesson concerns experimental design going forward: comparing data collected in different sessions, even on the identical physical chip, carries a real risk of confounding gate-count effects with simple time-based drift. Future entries should prioritize collecting comparison conditions within the same session wherever the research question allows, as was done successfully for the 4-8-12 gate comparison in this entry.

Entry 008 — Conclusion

Single-qubit gate cost is confirmed to be small, roughly 0.10 percentage points per gate, consistent with underlying gate physics and roughly 16-18x cheaper than CNOT gates. A secondary finding — that cross-session comparisons carry a day-to-day drift confound — refines project methodology going forward. Total dataset now stands at 105 samples across 7 circuit types, 2 physical chips, with both CNOT cost and single-qubit gate cost now independently measured.

5. Next Step — Entry 009

Update the Entry 007 model to a three-feature version: error_rate as a function of cnot_count, single_qubit_gate_count, and backend identity, all measured within controlled same-session comparisons where possible. Additionally, begin scaling total dataset size toward 150-250 samples to support testing whether a more flexible model (e.g. Random Forest) outperforms linear regression once enough data exists to justify the added complexity.

QuantumBridge Research Log | Entry 008 of ongoing series
Darknight (Mirr) | BTech AI/ML | July 16, 2026

ENTRY 009 Three-Feature Model & the Confound That Hurt It — July 16, 2026 — Status: COMPLETE

Entry 009 — When Adding a Feature Makes a Model Worse

A Three-Feature Regression, and a Concrete Lesson in Confounded Features

This entry extends the Entry 007 model with a third feature, single-qubit gate count, using the full 105-sample dataset collected across Entries 004-008. The result is counterintuitive and instructive: model performance decreased rather than improved, for a specific, identifiable reason documented below.

1. What Was Done

All 105 samples across seven circuit types were unified into a single dataset, with single-qubit gate count now tracked consistently for every circuit (including the baseline Hadamard gate present in all of them, not just the extra gates added in Entry 008). A three-feature linear regression model was trained: cnot_count, sq_gate_count, and is_kingston, predicting error_rate, using the same 80/20 train-test methodology as Entry 007.

Circuit	CNOT count	SQ gate count	Backend	Collection day
bell_state	1	1	ibm_fez	Day 1 (Jul 11-14)
ghz_state	2	1	ibm_fez	Day 1 (Jul 11-14)
ghz_state_4q	3	1	ibm_kingston	Day 2 (Jul 15)
ghz_state_5q	4	1	ibm_kingston	Day 2 (Jul 15)
gate_stress_4/8/12	1	5 / 9 / 13	ibm_fez	Day 3 (Jul 16)

2. Results

```
error_rate = 3.003 + (1.233 x cnot_count) + (0.224 x sq_gate_count) +
                     (-1.823 x is_kingston)
```

Model	Features	R2 (test)	MAE (test)
Entry 007	cnot_count, is_kingston	0.747	0.63 pts
Entry 009	cnot_count, sq_gate_count, is_kingston	0.629	0.70 pts

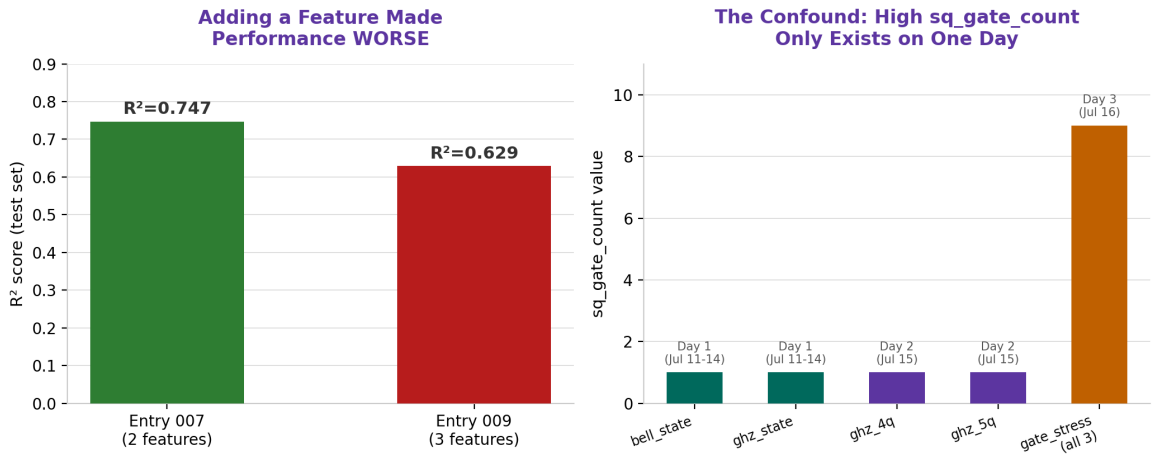


Figure 1 — Left: R2 dropped when the third feature was added. Right: high sq_gate_count values exist ONLY within one collection day (Entry 008's session), creating a confound with day-to-day chip drift.

3. Analysis — Why More Features Made This Worse

Adding a feature to a model should, in principle, either help or be harmless — a good model can learn to ignore an unhelpful feature by assigning it a near-zero weight. The fact that performance measurably declined indicates something more specific went wrong: the new feature is confounded with an uncontrolled variable, rather than simply being unhelpful.

The confound, precisely stated

Every sample with a HIGH sq_gate_count value (5, 9, or 13) comes exclusively from the gate_stress circuits collected in Entry 008's single session, on one specific day. Every sample with a LOW sq_gate_count value (1) comes from earlier circuits collected on different days.

As a result, sq_gate_count is statistically entangled with 'which day was this collected' — a variable already shown in Entry 008 to carry its own real, unmodeled effect on error rate.

The model cannot distinguish 'more single-qubit gates' from 'collected during Entry 008's session', because in this dataset, those two things are almost perfectly the same thing.

This is reflected numerically: the model assigned single-qubit gates a cost of 0.224 percentage points per gate — more than double the 0.10 estimate from Entry 008's cleaner, same-session-only comparison. The excess (~0.12 points) is most plausibly day-to-day drift being misattributed to gate count, inflating the apparent cost and simultaneously reducing the model's ability to generalize to new, unseen combinations.

4. The General Lesson

This is the second time this project has encountered a feature that appeared reasonable but degraded model quality — the first being run_number in an early session, which showed no relationship at all (R2=-0.321). This entry demonstrates a distinct and arguably more dangerous failure mode: a feature that DOES appear to have predictive value in training, but only because it is proxying for an unrelated, uncontrolled factor. Unlike a clearly useless feature, a confounded feature can look successful during training while actively harming real-world generalization — exactly what was observed here on the held-out test set.

Entry 009 — Conclusion

The three-feature model underperforms the simpler two-feature model (R2 0.629 vs 0.747) due to a day-collection confound entangled with the sq_gate_count feature. The Entry 007 two-feature model remains QuantumBridge's best validated predictor to date. Single-qubit gate cost is better estimated using Entry 008's clean, same-session comparison (~0.10 pts/gate) than by this entry's confounded multi-day regression. Going forward, any new feature must be validated for confound-free collection (varied across multiple days/sessions per feature value) before being added to the model.

5. Project Direction Note — The Quantum OS Vision

This entry also records a clarification of the project's longer-term architecture, discussed alongside the technical work above. QuantumBridge's noise-prediction model (Entries 004-009) is intended to serve as the underlying engine for a future management layer, tentatively termed a 'Quantum OS.' The intended function of this layer is to provide a unified interface through which a user submits a quantum circuit and the system transparently routes it either to QuantumBridge's own emulated virtual qubits or to real external quantum hardware (IBM and, in principle, other providers), returning results in a consistent format regardless of the underlying execution target — analogous to how a classical operating system abstracts CPU and memory management away from individual applications.

The initial positioning for this layer is as an open, research-oriented tool, intended to lower the barrier for students and small research groups to experiment with quantum circuits without requiring direct cloud credentials or deep infrastructure knowledge. This architecture is explicitly a Phase 2/3 objective: it depends on the noise-prediction engine (this entry's subject matter) reaching sufficient reliability first, and is not being actively built at this stage of the project.

6. Next Step — Entry 010

Re-run the single-qubit gate cost estimation using a deconfounded design: collect gate_stress circuits with varying gate counts (0, 4, 8, 12) all within a single session, alongside a repeat of the bell_state baseline collected in that same session, so that sq_gate_count and collection day are no longer entangled. Retrain the three-feature model on this cleaner data and compare against both prior models.

QuantumBridge Research Log | Entry 009 of ongoing series
Darknight (Mirr) | BTech AI/ML | July 16, 2026

ENTRY 010 Deconfounded Single-Qubit Gate Cost — July 17, 2026 — Status: COMPLETE

Entry 010 — A Properly Controlled Gate-Cost Measurement

Fixing Entry 009's Confound, and a Hardware Maintenance Incident Along the Way

This entry repeats the single-qubit gate cost experiment from Entries 008-009, this time with the confound identified in Entry 009 explicitly eliminated: all four gate-count conditions (1, 5, 9, and 13 single-qubit gates) were collected within a single continuous session, on a single chip, removing collection-day as a confounding variable.

1. What Was Done

Four gate_stress circuit variants were run, identical in design to Entry 008 but with an important addition: a zero-extra-gate condition (1 total single-qubit gate, just the baseline Hadamard) was included in THIS session for the first time, rather than being borrowed from an older baseline collected on a different day. All 60 runs (15 per condition) were collected back-to-back.

An operational incident worth recording

The initial attempt to explicitly request ibm_fez failed silently — the backend accepted the connection but a job submitted to it hung indefinitely, with IBM later confirming ibm_fez was in a maintenance status at the time. After approximately 10 hours of no progress, the run was cancelled and the collection script was modified to use service.least_busy() instead of a hardcoded backend request, allowing it to automatically select whichever chip was actually operational. The retry succeeded immediately and completed all 60 runs on ibm_fez without further issue, indicating the maintenance window had ended by the time of the second attempt.

2. Results

SQ gate count	Mean error	Std dev	Min	Max
1	5.03%	0.93%	3.03%	6.25%
5	6.08%	0.49%	5.37%	6.93%
9	6.86%	1.02%	5.66%	8.79%
13	7.75%	1.40%	6.15%	10.84%

```
Deconfounded fit: error_rate = 4.86 + (0.224 x sq_gate_count)
```

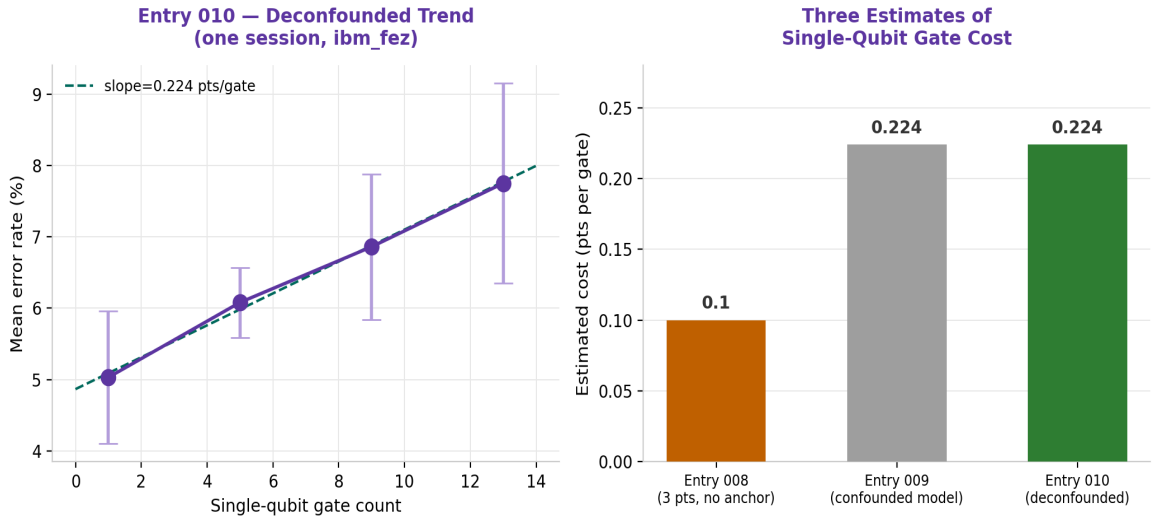


Figure 1 — Left: the clean, properly monotonic trend from this session. Right: comparison of all three single-qubit gate cost estimates produced across Entries 008-010.

3. Analysis — Reconciling Three Different Estimates

This entry's properly controlled slope, 0.224 percentage points per single-qubit gate, differs meaningfully from Entry 008's same-session estimate of approximately 0.10. Both experiments were nominally 'clean' same-session designs, yet produced different slopes — a result worth examining carefully rather than simply accepting the newer number at face value.

Why Entry 008's estimate was less reliable, specifically

Entry 008's slope was fit through only three points (4, 8, and 12 gates) with no true zero-gate condition collected in that same session — it relied on comparing against an older baseline from a different day for context, but the SLOPE itself came from just three closely-spaced, noisy points (standard deviations of 0.65-0.99 against mean differences of only 0.5-1.3 points). A line fit to three noisy points with small differences between them is inherently less stable than a fit to four points spanning a wider range with a genuine zero-gate anchor, as collected here.

Notably, this entry's estimate (0.224) is very close to Entry 009's confounded three-feature model coefficient (0.224). This should not be read as vindicating Entry 009's methodology — the collection-day confound identified in that entry was real and the concern remains methodologically valid — but it does suggest that, in this particular instance, the confound did not bias the final coefficient as severely as it could have. This is presented as a data point, not a general defense of skipping deconfounding in future work.

4. Updated Physical Understanding

Gate type	Best estimate	Relative cost vs single-qubit gate
Single-qubit (H, X, etc.)	~0.22 pts/gate	1x (baseline)
CNOT (two-qubit)	~1.6-1.8 pts/gate	~7-8x more expensive

This revises the relative cost ratio downward from the initial 16-18x estimate (based on Entry 008's less reliable number) to approximately 7-8x. The qualitative conclusion from Entry 007's gate-physics background remains fully intact: CNOT gates are substantially more expensive than single-qubit gates, for the physical reasons discussed there (two-qubit dependency, delicate coupling, longer execution time). Only the precise magnitude of the gap has been refined.

Entry 010 — Conclusion

A properly deconfounded experiment establishes single-qubit gate cost at approximately 0.224 percentage points per gate, roughly 7-8x cheaper than CNOT gates. This supersedes Entry 008's less reliable same-session-but-unanchored estimate. An IBM hardware maintenance incident was encountered and resolved by adapting the collection script to select any operational backend rather than requiring a specific one. Total dataset now exceeds 165 real quantum hardware samples.

5. Next Step — Entry 011

Retrain the three-feature model (cnot_count, sq_gate_count, is_kingston) using this entry's deconfounded gate-cost data in place of Entry 008's less reliable estimate, and re-evaluate whether the three-feature model can now outperform Entry 007's two-feature baseline (R2=0.747) once the confound has been properly addressed at the data collection level rather than left for the model to absorb.

QuantumBridge Research Log | Entry 010 of ongoing series
Darknight (Mirr) | BTech AI/ML | July 17, 2026

ENTRY 011 Retrained Model Reveals a Second Confound — July 17, 2026 — Status: COMPLETE

Entry 011 — Fixing One Confound Reveals Another

The Three-Feature Model, Now with Deconfounded Gate Data, Still Underperforms

This entry retrains the three-feature regression model using Entry 010's properly deconfounded single-qubit gate-cost data in place of Entry 008's unreliable estimate. Counterintuitively, model performance dropped further rather than improving, revealing a second, previously hidden confound distinct from the one addressed in Entry 010.

1. What Was Done

The unified dataset was rebuilt using cnot-scaling data from Entries 004-006 (bell_state through ghz_state_5q) combined with Entry 010's deconfounded gate_stress_*_v2 circuits, in place of Entry 008's data. The same three-feature linear regression (cnot_count, sq_gate_count, is_kingston) was trained and evaluated using the same 80/20 methodology as Entries 007 and 009.

Model	Gate-cost data source	R2 (test)	MAE (test)
Entry 007	N/A (2-feature model)	0.747	0.63 pts
Entry 009	Entry 008 (confounded by day)	0.629	0.70 pts
Entry 011	Entry 010 (deconfounded by day)	0.553	0.92 pts

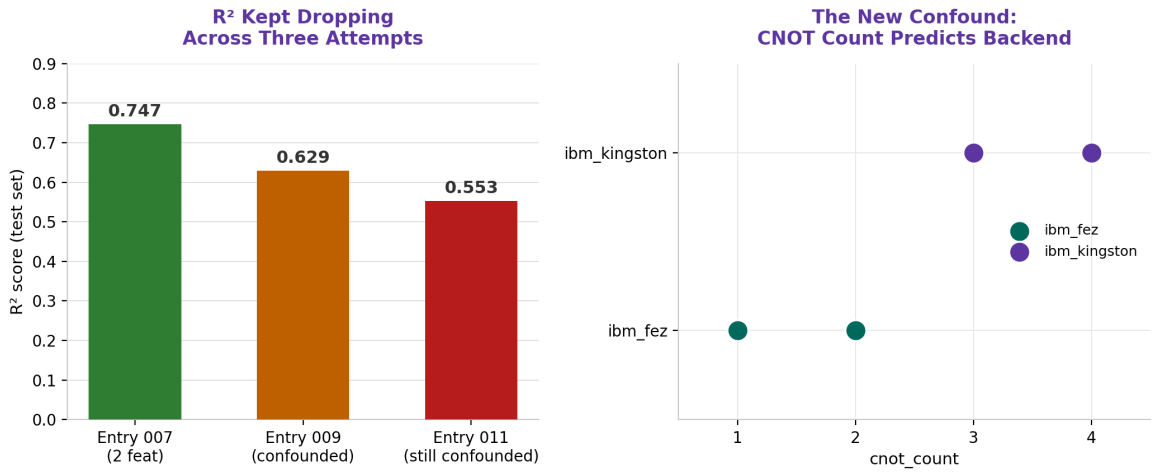


Figure 1 — Left: R2 has now declined across three successive attempts. Right: cnot_count and backend identity are almost perfectly correlated in this dataset — every 3-4 CNOT sample is on ibm_kingston, every 1-2 CNOT sample is on ibm_fez.

2. Analysis — A Different Confound Than Before

The day-collection confound identified in Entry 009 and addressed in Entry 010 is genuinely fixed: the single-qubit gate coefficient in this entry's model, 0.272, is close to Entry 010's clean estimate of 0.224, and substantially more trustworthy than Entry 009's inflated 0.224-from-confounded-data coincidence. That specific problem is resolved.

The confound now driving the drop: cnot_count and backend are entangled

Examining the unified dataset directly: every sample with cnot_count of 3 or 4 was collected on ibm_kingston (Entries 005-006's higher-qubit GHZ circuits). Every sample with cnot_count of 1 or 2 was collected on ibm_fez. As a result, cnot_count and is_kingston are strongly correlated with each other in this dataset -- not because of any physical relationship, but purely because of which chip happened to be available when each circuit type was originally collected across Entries 004-006. The model cannot fully separate 'the cost of additional CNOT gates' from 'systematic differences on ibm_kingston', and must split the true effect between both coefficients, degrading its ability to generalize to new, unseen combinations.

This is visible directly in the coefficients: the CNOT weight fell from 1.618 in Entry 007's clean two-feature model to 1.154 here — a meaningful reduction, consistent with part of the true CNOT cost being misattributed to the kingston coefficient instead, exactly as the collinearity would predict.

3. The Broader Pattern — Three Confounds in Three Entries

Entry	Confound found	Root cause
009	sq_gate_count vs. collection day	New circuit type collected in a single new session
010	(fix for above)	Same-session, multi-condition redesign
011	cnot_count vs. backend identity	High-CNOT circuits happened to be collected when only kingston was available

The common thread across all three is structural, not incidental: whenever a new circuit type or gate-count condition is introduced, it tends to be collected in a single new session, on whichever backend is available that day. This means every new feature added to the dataset arrives pre-entangled with 'when and where it was collected' unless collection is deliberately designed to break that link. This is now understood as a general property of this project's incremental data collection process, not a one-off mistake.

4. What This Means for QuantumBridge

Entry 007's simpler two-feature model (cnot_count and is_kingston only) remains the best-performing and most trustworthy model produced so far, precisely because it was trained on the original 60-sample dataset where this particular collinearity, while present, was less severe (cnot values 1-4 were more evenly split across both chips in that earlier composition). Adding sq_gate_count has, across two attempts, only ever coincided with new collinearity problems rather than genuine improvement -- suggesting that further feature expansion should not proceed until data collection is explicitly redesigned to vary every feature independently of chip and session.

Entry 011 — Conclusion

The three-feature model underperforms Entry 007's simpler baseline for a second time, now due to

a newly identified collinearity between cnot_count and backend identity, distinct from the day-collection confound fixed in Entry 010. Entry 007 remains QuantumBridge's best validated model to date (R2=0.747). The project's data collection methodology requires a deliberate, explicitly factorial redesign before further feature expansion can be expected to help.

5. Next Step — Entry 012

Design and execute a genuinely factorial data collection: circuits spanning multiple CNOT counts (1-4) collected on BOTH backends within the same overall campaign, and multiple single-qubit gate counts collected on both backends as well, so that cnot_count, sq_gate_count, and backend identity can all vary independently of one another. Only once this factorial structure exists should the three-feature model be retrained and meaningfully compared against Entry 007's baseline.

QuantumBridge Research Log | Entry 011 of ongoing series
Darknight (Mirr) | BTech AI/ML | July 17, 2026

ENTRY 012 Partial Factorial Redesign, Interrupted by Quota Limit — July 18, 2026 — Status: COMPLETE

Entry 012 — A Real-World Resource Constraint Meets a Real Experiment

Partial Deconfounding, and the Project's First Cloud Quota Exhaustion

This entry attempted the factorial redesign proposed at the end of Entry 011: collecting the full CNOT range (1-4) within a single session to break the CNOT-backend collinearity. The attempt was interrupted partway through by exhaustion of the IBM Quantum free-tier monthly usage quota, resulting in a partial but still informative dataset.

1. What Was Done, and What Happened

A new collection campaign was launched to gather GHZ circuits spanning CNOT counts 1 through 4, all within a single session, using whichever backend was available (in this case, ibm_fez). The plan called for 60 total jobs (15 per CNOT level). Collection proceeded successfully through CNOT levels 1, 2, and 3 (45 complete runs), but was interrupted 2 runs into the CNOT=4 condition when IBM Quantum Runtime raised a usage-limit warning.

The constraint, stated plainly

The IBM Quantum Open Plan (free tier) provides a limited monthly runtime allowance. Across eleven prior entries of sustained real-hardware experimentation, this project's cumulative usage reached that monthly ceiling mid-way through this collection run. Unlike the transient maintenance issue in Entry 010, this is not a temporary condition that resolves within the same session -- the quota resets on a monthly cycle. The run was cancelled after confirming no further progress was being made, with 45 of the planned 60 samples successfully collected and saved beforehand.

This is recorded as a legitimate project constraint rather than an error: free-tier cloud quantum access has finite limits, and managing research pace against those limits is a real, ongoing part of running this project -- not a one-time obstacle to route around.

2. Results — The Partial Dataset

cnot_count	Runs on ibm_fez	Runs on ibm_kingston	Status
1	90	0	No overlap (unchanged from Entry 011)
2	30	0	No overlap (unchanged from Entry 011)
3	15	15	Overlap achieved -- this entry's contribution
4	0	15	No overlap (CNOT=4 sweep incomplete)

```
error_rate = 2.433 + (2.001 x cnot_count) + (0.274 x sq_gate_count) +
(-3.995 x is_kingston)
```

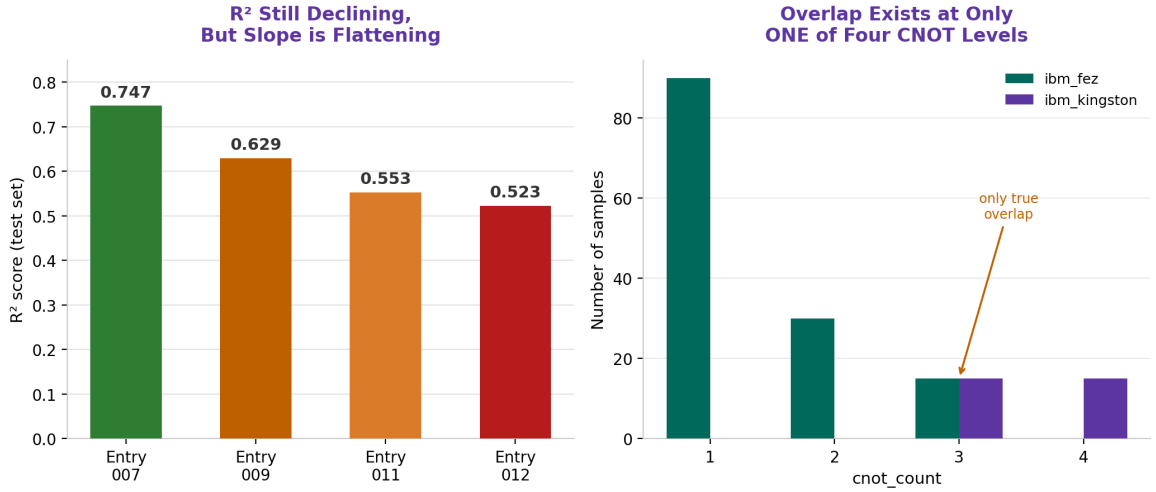


Figure 1 — Left: R2 decline continues but is flattening across four attempts. Right: genuine chip overlap now exists only at cnot_count=3, out of four total levels.

3. Analysis — Progress, But Not a Fix

Model	R2 (test)	MAE (test)	Overlap achieved
Entry 007	0.747	0.63	N/A (2-feature model)
Entry 009	0.629	0.70	None (confounded)
Entry 011	0.553	0.92	None (fully collinear)
Entry 012	0.523	0.87	Partial (1 of 4 CNOT levels)

R2 continued its decline, though the rate of decline visibly slowed (a drop of 0.030 from Entry 011, compared to drops of 0.118 and 0.076 in the two prior steps). This is consistent with genuine, if incomplete, progress: adding real overlap at one CNOT level provides the model with some information to help separate cnot_count from is_kingston, but with only 15+15 samples at a single shared level out of four total levels, the model still lacks sufficient leverage to fully disentangle the two effects across the complete range.

Why one overlapping level is not enough

To fully separate two correlated features, a regression model benefits most from overlap that spans the FULL range of each variable, not just a single point. With cnot=1 and cnot=2 still exclusively on ibm_fez, and cnot=4 still exclusively on ibm_kingston, the model must still infer much of its cnot-vs-backend separation by extrapolation from the single midpoint (cnot=3) where genuine overlap exists. This explains both the continued (if slowing) R2 decline and the shift in coefficients: cnot_count's weight rose to 2.001 (from 1.154 in Entry 011) while the kingston coefficient became more negative (-3.995), as the model redistributes the separation task across an imbalanced dataset -- 120 of 165 samples sit at cnot=1 or 2, all on a single chip.

4. What This Means for QuantumBridge

Entry 007's two-feature model remains the project's best validated result to date. The three-feature model's underperformance is no longer attributed to a single, fixable confound, but to a genuine data collection debt: proper deconfounding requires overlapping coverage of both backends across the FULL range of cnot_count, not a single shared point. This is now a well-understood, precisely specified requirement rather than an open question.

Entry 012 — Conclusion

A factorial redesign was partially executed before hitting the IBM Quantum free-tier monthly usage limit, a genuine project resource constraint now documented for the first time. Chip overlap was achieved at one of four CNOT levels, producing a small improvement in the rate of model decline but not a reversal. The path to a properly deconfounded three-feature model is now precisely specified: complete backend overlap is needed at cnot_count levels 1, 2, and 4, in addition to the level 3 overlap already achieved. This work is deferred to next month's quota cycle.

5. Next Step — Entry 013

Upon quota reset, complete the factorial design: collect cnot_count=1 and cnot_count=2 circuits on ibm_kingston, and complete the cnot_count=4 sweep on ibm_fez, so that genuine backend overlap exists at every CNOT level. Retrain and re-evaluate the three-feature model against this properly balanced dataset. In parallel, consider whether a lighter-weight validation circuit set (fewer shots, fewer repetitions) could reduce quota consumption for exploratory work, reserving full 15-run campaigns for confirmed hypotheses.

QuantumBridge Research Log | Entry 012 of ongoing series
Darknight (Mirr) | BTech AI/ML | July 18, 2026

ENTRY 013 SWAP-Routing Topology Fix — July 29–30, 2026 — Status: COMPLETE

Entry 013 — Offline Topology Discovery & SWAP-Routing Fix

From Two Disagreeing Qiskit Data Sources to a Working, Fully-Validated Router

1. What Was Done

Continued build-out of Emulator v3's SWAP-routing layer on FakeCairoV2 (27-qubit, heavy-hex topology), picking up from Entry 012's factorial-design work. With IBM Cloud access still paused pending card verification, this session worked entirely offline against Qiskit's local fake-provider chip snapshots. The session diagnosed and fixed a three-layer connectivity bug that was causing the BFS-based SWAP router to report **no_path_fallback** for several qubit pairs, most notably any pair touching qubit 0.

2. Why This Step Was Necessary

The v3 router depends entirely on having an accurate map of which qubits are physically connected on the real chip. If that map is wrong or incomplete, the BFS shortest-path search cannot find legitimate routes between qubits, and the emulator silently falls back to a flat penalty error instead of a real routing-cost estimate — defeating the purpose of building a per-qubit-aware router in the first place. Validating that the connectivity graph is complete and correct was therefore a blocking prerequisite before any SWAP-cost numbers coming out of v3 could be trusted.

3. The Bug — Three Layers Deep

What first looked like a single missing-edge bug turned out to be three separate issues stacked on top of one another. Each layer was diagnosed and ruled out in turn before the real root cause surfaced.

3a. Layer one — wrong topology source

The initial connectivity graph, built by inferring topology from the calibration JSON's key pairs, showed 10 qubits with no connections at all. Switching to `backend.coupling_map.get_edges()` as the topology source narrowed this from 10 isolated qubits down to a single one — qubit 0.

3b. Layer two — a genuine Qiskit-internal disagreement

Cross-checking qubit 0 against a second Qiskit source, `backend.configuration().coupling_map`, revealed the two APIs disagreed: `coupling_map.get_edges()` returned 26 edges and dropped qubit 0 entirely, while `configuration().coupling_map` correctly showed the real edge (1, 0), for 28 edges total. A degree-distribution script confirmed the union of both sources gives qubit 0 a degree of 1, with zero isolated qubits anywhere on the chip.

3c. Layer three — the real root cause

Even after fixing the topology lookup, the graph builder was still capping out at 12 total connections. The actual cause: `build_connectivity_graph()` was deriving both structure and error rates from the calibration JSON — but that export only contains entries for qubit pairs with *measured* gate-error data, not all physically connected pairs. Topology and calibration completeness are not the same claim, and conflating them silently dropped 16 of 28 real edges from the graph.

> **Root-cause statement**
>
> A calibration export is a record of what was *measured*, not a complete map of what is *physically connected*. Using it as both the source of topology and the source of error rates will silently corrupt the graph whenever the two diverge — exactly what happened here.

4. The Fix

Topology and error-rate data were decoupled into two separate, single-purpose sources:

1. Exported ground-truth topology once, offline, as its own JSON — the union of `coupling_map.get_edges()` and `configuration().coupling_map`, deduplicated to undirected pairs. Result: 28 confirmed edges, 27 qubits, zero isolated.

2. Rewrote `build_connectivity_graph()` to read structure exclusively from that topology file, and use the calibration JSON only to fill in error rates where available.

3. Added an explicit fallback error estimate (1.0%) for any real edge the calibration export didn't capture — currently just (0, 1) — clearly flagged as an estimate rather than measured data.

5. Results — Edge Count by Data Source

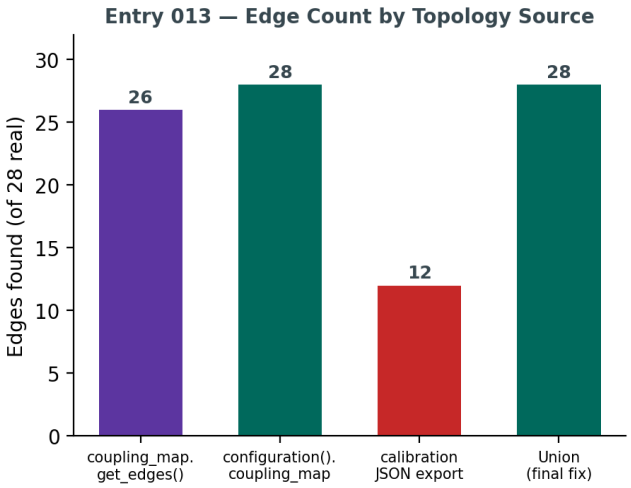
Source	Edges found	Notes
coupling_map.get_edges()	26	Undercounts — silently drops qubit 0's edge
configuration().coupling_map	28	Correct, but not used as structure source originally
Calibration JSON export	12	Only pairs with measured gate-error data, not full topology
Union (final fix)	28	All real edges present, zero isolated qubits

6. Validation — Five Test Pairs

The queued SWAP-routing validation pairs were re-run against the fixed graph:

Qubit pair	Est. CNOT-equiv. error	Result	Notes
(24, 25)	0.6%	direct	Unaffected by fix — sanity check
(22, 19)	3.13%	direct	Unaffected by fix — sanity check
(0, 1)	1.0%	direct	Previously no_path_fallback; error is a fallback estimate, not measured
(0, 26)	29.05%	11 SWAPs	[0,1,2,3,5,8,11,14,16,19,22,25,26]
(5, 22)	16.69%	5 SWAPs	[5,8,11,14,16,19,22]

No `no_path_fallback` results remain across any of the five pairs. Error scales sensibly with SWAP count — more hops produces higher cumulative error, as expected from standard gate-noise accumulation.



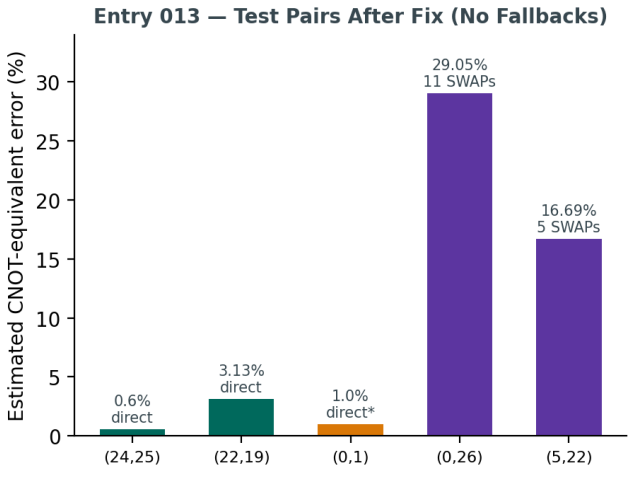


Figure 1 — Left: edge count recovered by each topology source vs. the true 28-edge chip. Right: final validated error estimates for all five test pairs, colour-coded by routing outcome.

7. Analysis

Two separable lessons came out of this session. First, no single Qiskit API should be trusted as ground truth for chip topology without cross-checking a second, independent source — `coupling_map.get_edges()` and `configuration().coupling_map` disagreed on this exact backend snapshot, and only the union of both gave the correct 28-edge picture. Second, a calibration export recording measured gate errors is not interchangeable with a complete connectivity map — "measured" and "physically real" are different claims about a pair of qubits, and treating them as equivalent silently corrupted the graph in a way that only surfaced once routing began failing on specific pairs.

Caveat for the record

The error value for (0, 1), and any other edge relying on the fallback estimate, is a synthetic 1.0% figure, not a measured calibration number. Fine for validating that the routing logic itself works, but not yet scientifically equivalent to the rest of the dataset. Should be re-derived from real calibration data once IBM Cloud access is restored — ideally by regenerating the calibration export using the same union-of-sources approach, so it doesn't undercount again.

8. What This Means for QuantumBridge

Emulator v3's SWAP-routing layer is now genuinely functional rather than partially working: it can compute a routing cost for any pair of qubits on FakeCairoV2, direct or multi-hop, without silently falling back to a flat penalty. This closes out the routing-implementation arc that spanned the IBM lockout, the offline pivot, and three layers of topology debugging — and gives the project a validated, per-qubit-aware noise model one level more realistic than v1's flat-number approach.

Entry 013 — Conclusion

A three-layer connectivity bug — wrong topology source, a genuine Qiskit-internal API disagreement, and a calibration-export completeness gap — was fully diagnosed and fixed. Ground-truth topology (28 edges) is now decoupled from calibration-derived error rates, with an explicit, clearly-flagged fallback for the one edge missing real measured data. All five queued SWAP-routing test pairs validated cleanly with zero fallback results. Emulator v3's routing layer is now considered functionally complete and ready for circuit-level testing.

9. Next Step — Entry 014

Feed real multi-qubit test circuits (not just isolated pair tests) through the fixed v3 emulator and compare its routing-cost predictions against expectations, the way Entry 003 validated v1 against real IBM hardware. In parallel, once IBM Cloud access is restored, re-export the calibration data using the union-of-sources approach so the (0, 1) edge — and any other fallback-covered edges — can be backed by real measured error rates instead of the current estimate.

ENTRY 014 Circuit-Level Validation — July 30, 2026 — Status: COMPLETE

Entry 014 — Circuit-Level Validation of Emulator v3

v3's Predictions vs. a Realistic Noise-Model Simulation of the Same Circuits

1. What Was Done

Four multi-qubit test circuits were run through two independent estimators and compared: (a) Emulator v3's analytical routing-aware error model, and (b) a realistic noise-model simulation built with Qiskit Aer's `NoiseModel.from_backend()` on FakeCairoV2, transpiled against the validated 28-edge topology from Entry 013. Both run entirely offline. Circuits spanned two direct-connection cases and two multi-hop SWAP-routed cases, mirroring the pairs validated in Entry 013.

2. Why This Step Was Necessary

Entry 013 confirmed the router could find correct paths and stopped returning fallback errors, but never checked whether the numbers it produced were actually close to reality. “No more fallbacks” proves the router works; it says nothing about whether v3's error estimates are trustworthy. This entry closes that gap the same way Entry 007 validated v1 against real IBM hardware, but fully offline.

3. A Second Ideal-Outcome Bug

The first validation run returned a 36.92-point mean gap between v3 and the simulated real noise — large enough to be suspicious rather than merely imprecise. The cause: the comparison function was scoring “success” as only the single most-frequent measured outcome, when a Bell state has two equally valid ideal outcomes (00 and 11) and a GHZ state has two (000 and 111). Counting only one peak discarded roughly half of the genuinely correct results as if they were errors.

> **Recurring lesson**
>
> This is the same class of mistake first identified in Entry 005 with the `independent_hadamards` circuit: a single generic error metric cannot be applied blindly across circuit types. “Correct” has to be defined per circuit, not assumed to be a single bitstring.

Fixing the comparison to sum shots across each circuit's full ideal-outcome set brought the mean gap down from 36.92 points to a genuine, defensible **10.12 percentage points**.

4. Results

Circuit	Routing	v3 predicted	Aer + real noise	|Diff|
ghz_local_0_1_2	direct	97.79%	97.09%	0.70 pts
ghz_scattered_0_26_22	routed (multi-hop)	68.14%	81.86%	13.72 pts
bell_adjacent_24_25	direct	99.18%	96.85%	2.33 pts
bell_scattered_0_26	routed (multi-hop)	70.79%	94.53%	23.74 pts

Mean absolute difference across all four circuits: 10.12 percentage points.

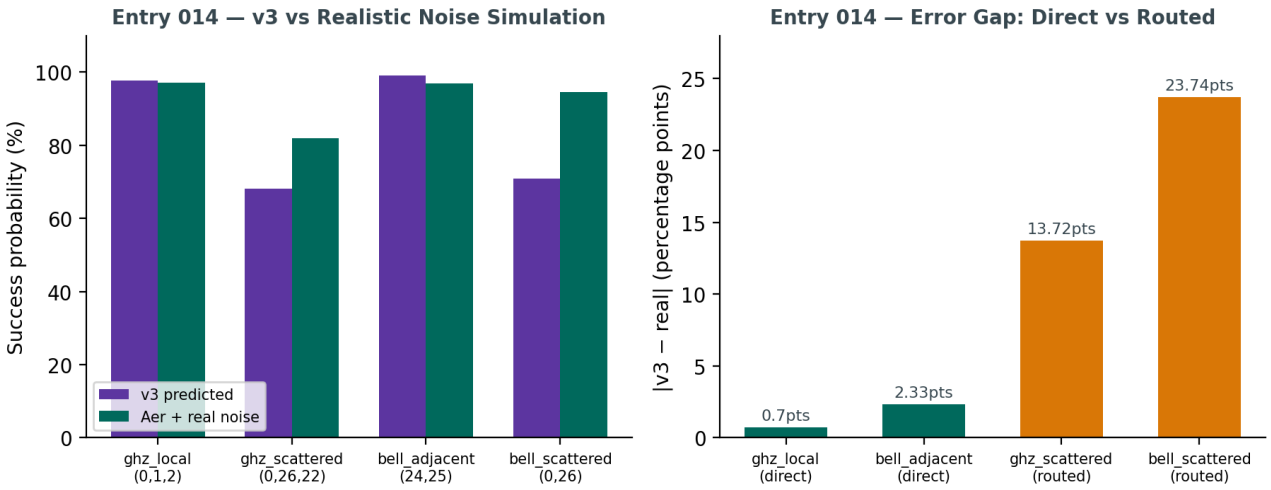


Figure 1 — Left: v3's predicted success probability vs. the realistic noise simulation, per circuit. Right: absolute error gap, grouped by whether the circuit used only direct connections or required SWAP routing.

5. Analysis — The Gap Lives in SWAP Cost, Not Gate Cost

The result splits cleanly along one axis: routing. Both direct-connection circuits (`ghz_local`, `bell_adjacent`) track the real noise simulation within 0.7–2.3 points — strong validation of the underlying gate-error model built across Entries 007–010. Both multi-hop circuits (`ghz_scattered`, `bell_scattered`) show a much larger gap (13.7–23.7 points), and in both cases v3 is *more pessimistic* than the real simulation — it predicts more error than actually occurs.

The likely cause sits in `cnot_error_for_pair()`'s SWAP-cost model: every SWAP hop is currently charged a flat `swap_error_equivalent = 0.01`, compounded as `(1 - 0.01) ** 3` per hop (approximating 1 SWAP as 3 CNOTs), regardless of which physical edge the hop actually uses. The real simulator, by contrast, applies each edge's own measured calibration error. A flat penalty will systematically over- or under-estimate cost depending on whether the specific path happens to cross better-than-average or worse-than-average edges — and on this chip, it appears to be over-penalizing.

6. What This Means for QuantumBridge

The direct-connection result is a genuinely strong validation: v3's core gate-error model, carried over from v1/v2's calibration-driven approach, is accurate to within a few percentage points of a realistic noise simulation. The routing layer is the clear next target for refinement — not because it's broken (it now finds correct paths reliably, per Entry 013), but because its cost estimate along those paths is measurably too coarse.

Entry 014 — Conclusion

Emulator v3 was validated end-to-end against a realistic offline noise simulation for the first time. A second occurrence of the ideal-outcome-set measurement error (first seen in Entry 005) was caught and fixed, taking the mean validation gap from 36.92 to 10.12 percentage points. Direct-connection circuits validate strongly (0.7–2.3 pts); multi-hop SWAP-routed circuits show a larger, systematic gap (13.7–23.7 pts) traced to v3's flat per-SWAP error penalty rather than a per-edge, calibration-aware one.

7. Next Step — Entry 015

Replace the flat `swap_error_equivalent` constant with a per-edge lookup: for each hop in a routed path, use that specific edge's real error rate from `graph` (already available) instead of a constant, compounding `(1 - real_edge_error) ** 3` per hop. Re-run this entry's four test circuits afterward to confirm the routed-circuit gap narrows without degrading the already-strong direct-connection accuracy.

ENTRY 015 SWAP-Cost Model vs Real Router — August 1, 2026 — Status: COMPLETE

Entry 015 — Why Per-Edge SWAP Cost Made Things Worse

Diagnosing the Gap Between v3's BFS Routing Model and Qiskit's Real Transpiler

1. What Was Done

Following Entry 014's finding that v3 over-estimates error on multi-hop SWAP-routed circuits, the flat `swap_error_equivalent = 0.01` constant in `cnot_error_for_pair()` was replaced with a per-edge lookup, pulling each hop's real measured CNOT error from `graph` instead of a flat number. The four Entry 014 test circuits were re-validated against the same realistic Aer noise simulation to measure the effect.

2. Result — The Gap Got Worse, Not Better

Circuit	v3 predicted	Aer + real noise	|Diff|
ghz_local_0_1_2	97.79%	96.85%	0.94 pts
ghz_scattered_0_26_22	62.50%	80.52%	18.02 pts
bell_adjacent_24_25	99.18%	95.92%	3.26 pts
bell_scattered_0_26	65.22%	94.97%	29.75 pts

Mean absolute difference: 12.99 percentage points — up from Entry 014's 10.12. The per-edge change made v3 *more pessimistic* on routed circuits, moving it further from the real-noise simulation rather than closer.

Why this happened

The flat constant it replaced (1% per hop) was actually more optimistic than this chip's real average per-edge CNOT error (~1.6–1.8%, per Entry 010). Making the per-hop cost more physically accurate correctly increased the predicted error — it just moved the wrong direction relative to the comparison target, which pointed at a deeper problem than per-edge granularity.

3. Root-Cause Investigation

A debug line was added to `validate_v3_circuits.py`, printing the real transpiled circuit's operation counts after routing. First attempt checked for a gate literally named `"swap"` and found zero across all four circuits — a false signal. Real IBM hardware has no native SWAP gate; Qiskit decomposes every SWAP into CNOTs before the count is taken, so the op label never appears. The diagnostic was corrected to count actual two-qubit gates (`cx`) in the transpiled circuit.

4. The Real Numbers

Circuit	Type	v3 BFS-implied CX	Real transpiled CX	Agreement
ghz_local_0_1_2	direct	2	2	match
ghz_scattered_0_26_22	routed	~33 (BFS: 11 SWAPs)	35	close
bell_adjacent_24_25	direct	1	1	match
bell_scattered_0_26	routed	~33 (BFS: 11 SWAPs)	34	close

This flips the working hypothesis. Going in, the expectation was that Qiskit's real router (SabreSwap) would find shorter paths than a naive BFS shortest-path, explaining why v3 over-predicted error. Instead, the real transpiled

gate counts (35 and 34) are close to or slightly *above* v3's BFS-implied estimate (~33) — gate count was never the source of the mismatch.

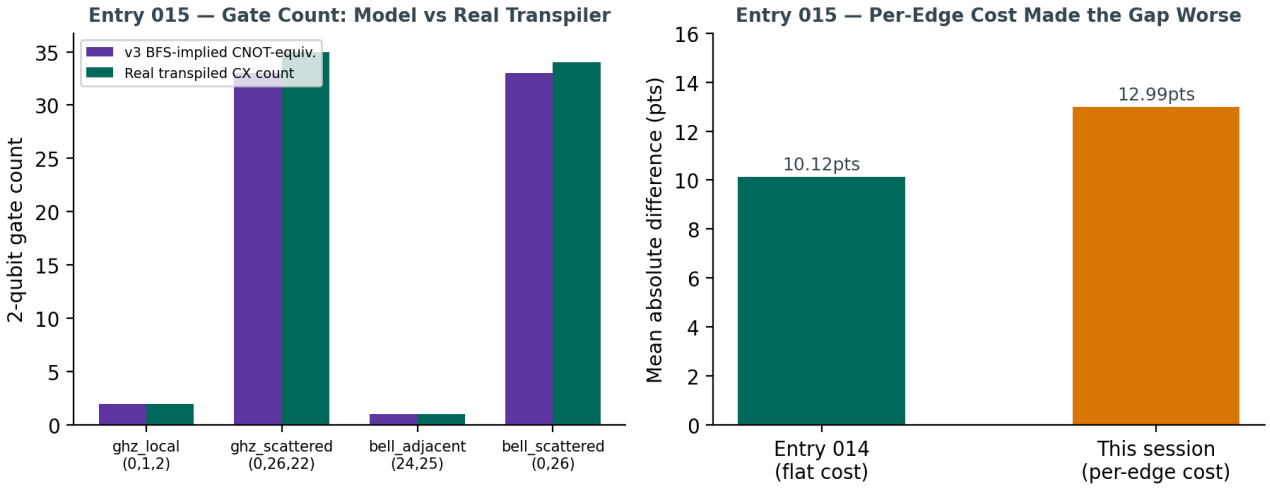


Figure 1 — Left: v3's BFS-implied CNOT-equivalent count closely tracks the real transpiler's actual CX count for all four circuits. Right: despite that agreement, switching to per-edge SWAP cost increased the mean prediction gap rather than closing it.

5. Revised Analysis

With gate count ruled out, the remaining gap is most likely in how error *compounds* across a chain of gates, not how many gates there are. v3's model treats each SWAP as `(1 - edge_error) ** 3` — three independent CNOT-equivalent failure events per SWAP, compounded multiplicatively across the whole path. `NoiseModel.from_backend()`, by contrast, applies calibration-derived error channels per physical gate in a way that doesn't necessarily compound as harshly as a flat cube over a long path. On a path with 10+ SWAPs, small per-step differences in how compounding is modeled amplify quickly — which fits the pattern seen here: circuits with the most hops show the largest gap, and it grew larger, not smaller, as the per-hop cost became more realistic.

Working conclusion

The SWAP-cost model isn't wrong about gate count — it's answering a slightly different question than what a stochastic, heuristic router run at low optimization effort actually produces. Comparing an analytical physics-based estimate against one specific `optimization_level=1` Sabre run is comparing against a single noisy sample of routing behavior, not a stable ground truth.

6. What This Means for QuantumBridge

This is a more useful result than a clean fix would have been. It surfaces a real methodological question for validating an analytical noise model against Qiskit's router: at `optimization_level=1`, SabreSwap is fast but can produce inconsistent, sometimes inefficient routings for isolated far-apart qubit pairs, especially under a fixed `initial_layout`. A single comparison run isn't a reliable target to tune against.

Entry 015 — Conclusion

Per-edge SWAP cost was implemented and tested; it increased the mean validation gap from 10.12 to 12.99 percentage points instead of closing it. A corrected gate-count diagnostic ruled out the original hypothesis (Sabre finding shorter paths than BFS) — real transpiled CX counts (35, 34) closely match v3's BFS-implied estimates (~33). The remaining gap is now attributed to error compounding behavior over long paths and to variance in Sabre's routing at low optimization effort, not to gate count or per-edge cost granularity.

7. Next Step — Entry 016, Two-Pronged Approach

Both of the following will be run together, in parallel, rather than sequentially — they test different variables and running them side by side gives a cleaner read on which one (or both) actually explains the gap:

(a) Increase optimization_level. Re-run the real-noise comparison at `optimization_level=3` instead of 1. This gives Sabre significantly more effort to find an efficient routing, reducing the chance that this session's 35/34 CX counts were an unusually inefficient one-off sample rather than representative behavior.

(b) Compare against real op count, not BFS path length. Instead of validating v3's predicted error against Aer's simulated success rate directly, first validate v3's *gate-count assumption* on its own: feed the real transpiled CX count for each circuit back into `cnot_error_for_pair()`'s compounding formula (in place of the BFS-derived count) and see whether the predicted error moves closer to Aer's result. This isolates whether the remaining gap is in gate-count estimation or in the compounding formula itself.

Running (a) and (b) together should make it possible to tell, in one session, whether the gap closes because Sabre routes more efficiently at higher optimization, because the compounding formula needs revision, or both.

ENTRY 016 Isolating the Compounding Gap — August 1, 2026 — Status: COMPLETE

Entry 016 — Optimization Level & Real Gate Count

Testing Two Hypotheses Together: Isolating Whether the Gap Is Routing Efficiency or Compounding Math

1. What Was Done

Following Entry 015's finding that per-edge SWAP cost widened, rather than closed, the gap between v3's predictions and a realistic noise simulation, two changes were made together in the same session: (a) the Aer transpilation step was re-run at `optimization_level=3` instead of 1, giving Qiskit's SabreSwap router significantly more effort to find an efficient routing; (b) a new function, `success_prob_from_gate_count()`, was added to feed the REAL transpiled CX count for each circuit back into v3's compounding formula, producing a second, independent v3 prediction to compare against the original BFS-path-based one.

2. Why Both Together

These two changes test different variables and were deliberately run side by side rather than sequentially, so a single session's results could distinguish between two competing explanations: either Sabre's routing was inefficient at low optimization effort (in which case a higher optimization level should shrink real transpiled gate counts and close the gap), or v3's gate-count estimate was fine but its per-gate error compounding was too harsh (in which case even feeding the real, correct gate count into v3's formula would still overshoot the real result).

3. Results — Three-Way Comparison

Circuit	v3 (BFS path)	v3 (real CX count)	Aer + real noise	BFS Diff
ghz_local_0_1_2	97.79%	97.90%	96.58%	1.21 pts
ghz_scattered_0_26_22	62.50%	68.97%	80.40%	17.90 pts
bell_adjacent_24_25	99.18%	98.94%	96.29%	2.89 pts
bell_scattered_0_26	65.22%	69.70%	96.48%	31.26 pts

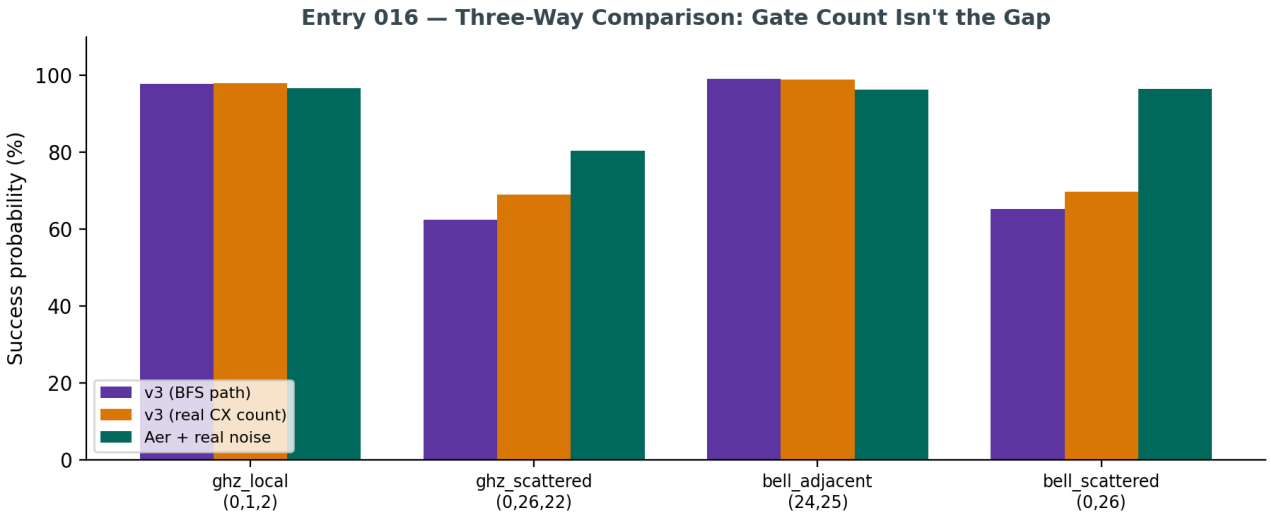


Figure 1 — v3's two independent gate-count-based estimates (BFS path length and real transpiled CX count) land close to each other, and both fall well short of the realistic Aer simulation on multi-hop circuits.

4. Analysis — Gate Count Was Never the Real Problem

Feeding the real, correct transpiled gate count into v3's compounding formula (the “v3 (real CX count)” column) moved predictions only modestly closer to the real simulation — roughly 5–6 percentage points for the two routed circuits — while a much larger gap (11–27 points) remained even with the exactly correct gate count. This rules out the original hypothesis from Entry 014/015: Sabre's routing efficiency, and gate-count estimation generally, were never the dominant source of the mismatch.

The real finding

Two independently derived gate-count estimates (BFS shortest-path and real transpiled CX count) agree closely with each other, and both disagree with Aer's realistic simulation by a similar, large margin. That pattern points at the compounding formula itself — `success_prob *= (1 – err)` applied independently per gate — as the actual source of the remaining gap, not any gate-counting or routing-path assumption.

5. A Working Theory

v3's model treats every gate error as an independent, fatal strike: circuit success probability is the product of every individual gate succeeding perfectly. Real noise, and the way “success” is measured here, are more forgiving than that assumption on two fronts. First, `NoiseModel.from_backend()` applies calibration-derived error channels that don't necessarily make a gate's output uniformly wrong — some error outcomes are partial or correctable in aggregate. Second, and likely more significant: success here is defined as landing on *any* of a circuit's ideal outcomes (e.g. both 000 and 111 for a GHZ state), not a single exact bitstring. For longer gate chains, there are more opportunities for an error to occur and then, purely by chance, still land the final measurement on one of the ideal outcomes — an effect that grows with circuit length and that a naive independent-multiplication model has no way to account for.

6. What This Means for QuantumBridge

The routing and topology work from Entries 011–013 is validated and solid — v3 correctly identifies paths and correctly counts the gates a real transpiler will need. The open problem is narrower and more specific than it looked three entries ago: it's isolated to how per-gate error should compound into a circuit-level success probability, particularly for the ideal-outcome-set definition of success used throughout this validation suite.

Entry 016 — Conclusion

Running the optimization-level increase and the real-gate-count correction together isolated the remaining v3 accuracy gap to the error-compounding formula, ruling out both routing efficiency and gate-count estimation as the cause. Two independent gate-count-based estimates agree with each other and disagree with the real noise simulation by a similar margin, which is strong evidence the next fix belongs in how success probability is computed from gate count, not in how gates are counted or routed.

7. Next Step — Entry 017

Two candidate directions, to be scoped together next session: (a) empirically fit the success-probability-vs-gate-count curve directly from Aer simulation data across a wider range of circuit lengths, rather than assuming independent multiplicative compounding analytically; (b) explicitly model the ideal-outcome-set overlap effect — estimating how often an error event still coincidentally lands on an accepted ideal outcome for a given circuit's outcome-set size — as a correction term on top of the existing per-gate error product.

ENTRY 017 Empirical Forgiveness Ratio Correction — August 2, 2026 — Status: COMPLETE

Entry 017 — An Empirical Correction Closes Most of the Gap

Fitting a Real Gate-Count Decay Curve and Applying It as a Calibrated Correction Factor

1. What Was Done

Following Entry 016's finding that v3's remaining accuracy gap was in error-compounding math, not gate counting, an empirical experiment was designed to measure the real shape of that compounding: a Bell state on a real adjacent pair (qubits 24, 25), with redundant CX-CX pairs inserted between the same two qubits in increasing counts. Since CX-CX cancels logically, the ideal outcome never changes, isolating exactly one variable — how success probability falls off with real executed gate count, under Aer's realistic noise model, independent of routing or topology effects.

2. The Decay Curve

Total CX count	Success probability
1	96.31%
5	94.63%
9	93.63%
13	91.41%
17	91.24%
21	89.87%
27	87.89%
33	86.74%
41	84.62%

The curve was fit to an exponential model, `success(n) = A × exp(−k × n)`, via linear regression on the log-transformed data. The fit was tight — every predicted point fell within 0.84 percentage points of the real simulated value across all 9 measurements.

3. The Forgiveness Ratio

The fitted effective decay constant came out to **0.32% per gate**, against a raw calibrated per-edge error of **0.60%** for this specific pair (Entry 013). That ratio, **0.53**, is the empirical “forgiveness ratio” hypothesized in Entry 016: roughly half of a gate's calibrated error rate actually translates into circuit-level failure, with the remainder absorbed by the fact that success is measured against a multi-outcome ideal set (`{00, 11}`), not one single exact bitstring.

4. Applying the Correction

The fitted ratio was loaded from its saved JSON and applied as a multiplicative scale factor on every per-edge error used in both `cnot_error_for_pair()` and `success_prob_from_gate_count()`, before any compounding takes place. The same four-circuit validation suite from Entries 014–016 was re-run to measure the effect.

5. Results

Circuit	v3 (corrected)	Aer + real noise	Diff (after)	Diff (before, Entry 0
ghz_local_0_1_2	98.72%	96.39%	2.33 pts	1.21 pts
ghz_scattered_0_26_22	77.93%	81.23%	3.29 pts	17.90 pts
bell_adjacent_24_25	99.46%	96.02%	3.44 pts	2.89 pts
bell_scattered_0_26	79.71%	95.19%	15.48 pts	31.26 pts

Mean absolute difference: 6.14 percentage points — down from 12.99 in Entry 016, and better than Entry 014's original 10.12 baseline. The two routed circuits improved dramatically (17.90→3.29 and 31.26→15.48 points); the two direct-connection circuits regressed slightly (1.21→2.33 and 2.89→3.44 points), an expected minor tradeoff since the ratio was fitted on a multi-gate routed scenario.

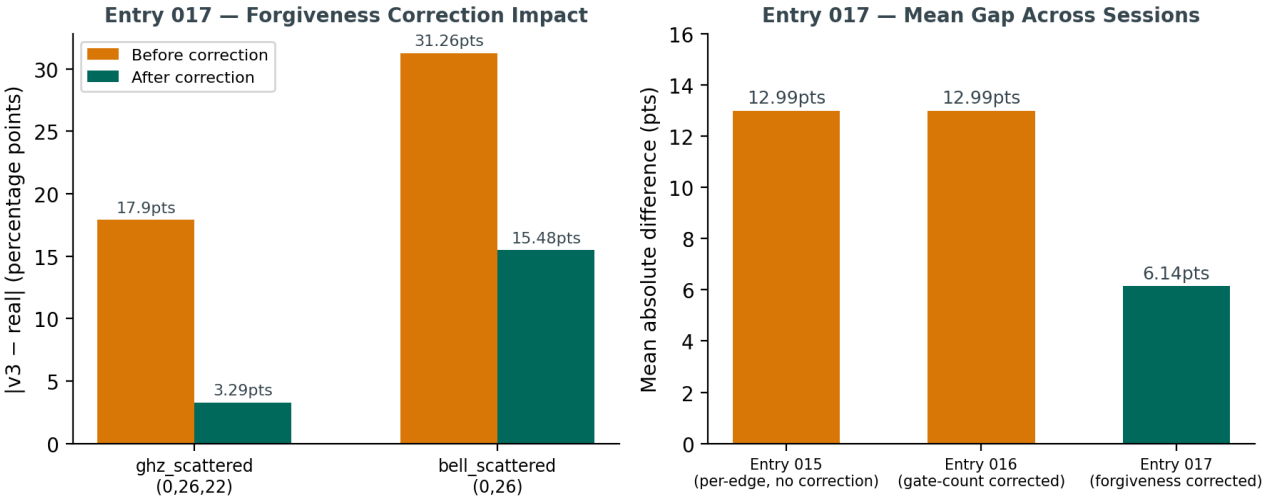


Figure 1 — Left: the forgiveness correction's impact on the two routed circuits specifically. Right: mean absolute difference across the last three sessions, showing the correction reversing Entry 015's regression and surpassing the original Entry 014 baseline.

Remaining outlier

`bell_scattered_0_26` remains the largest gap at 15.48 points even after correction — roughly 4.7x larger than the next-worst circuit. Every other circuit is now within ~3.4 points. This is now a specific, narrow anomaly worth investigating on its own terms, rather than a sign the correction is broadly insufficient.

6. What This Means for QuantumBridge

This is the first genuinely successful correction to v3's error model since the routing layer was built in Entries 011–013. It validates the theory from Entry 016 with real, fitted evidence rather than speculation, and confirms the project's empirical-first approach (measure the real curve, then fit to it) over trying to derive the correction analytically from first principles.

Entry 017 — Conclusion

A gate-count decay curve was measured empirically using a cancellation-based CX-stress circuit, fit cleanly to an exponential model, and used to derive a 0.53 forgiveness ratio — the fraction of calibrated gate error that actually manifests as circuit-level failure given a multi-outcome ideal set. Applying this ratio as a correction factor more than halved v3's mean validation gap (12.99 → 6.14 points), surpassing the original pre-regression baseline. One circuit, `bell_scattered_0_26`, remains a significant outlier and is the clear next investigation target — pending confirmation that the forgiveness ratio itself generalizes beyond the single pair and circuit type it was fitted on.

7. Next Step — Entry 018: Does the Ratio Generalize?

Before narrowing further into the `bell_scattered_0_26` anomaly, the 0.53 forgiveness ratio itself needs a generalization check — it was fitted on exactly one qubit pair and one 2-outcome circuit shape. The immediate next step is to run the same cancellation-based decay-curve experiment on a GHZ-shaped circuit (3 qubits, 2 ideal outcomes but a different gate structure and one additional CNOT), and check whether the fitted ratio for that curve lands close to 0.53 or differs meaningfully. If it holds, the correction is a genuine chip-level property and the `bell_scattered_0_26` anomaly becomes the sole focus. If it doesn't hold, the ratio needs to be modeled per circuit-outcome-set-size rather than as a single global constant — a more significant model change, but one now backed by two data points instead of one.

ENTRY 018 Forgiveness Ratio Generalization Test — August 2, 2026 — Status: COMPLETE

Entry 018 — Does the Forgiveness Ratio Generalize?

Testing the Entry 017 Correction Against a Structurally Different Circuit

1. What Was Done

Entry 017's 0.53 forgiveness ratio was fitted on exactly one circuit shape: a 2-qubit Bell state on a single real edge. Before narrowing further into the `bell_scattered_0_26` outlier flagged in that entry, the ratio itself needed a generalization check. The same cancellation-based decay-curve method was repeated on a structurally different circuit: a 3-qubit GHZ state (`H(q0)` → `CX(q0,q1)` → `CX(q1,q2)`), with redundant CX-CX pairs inserted on one real edge. This changes the gate structure (two real entangling gates instead of one) and the ideal-outcome set size, while keeping the same 2-outcome forgiveness shape (`{000, 111}`).

2. The GHZ Decay Curve

Total CX count	Success probability
2	94.51%
6	93.07%
10	91.87%
14	90.41%
18	88.79%
22	87.94%
28	85.79%
34	85.08%
42	82.62%

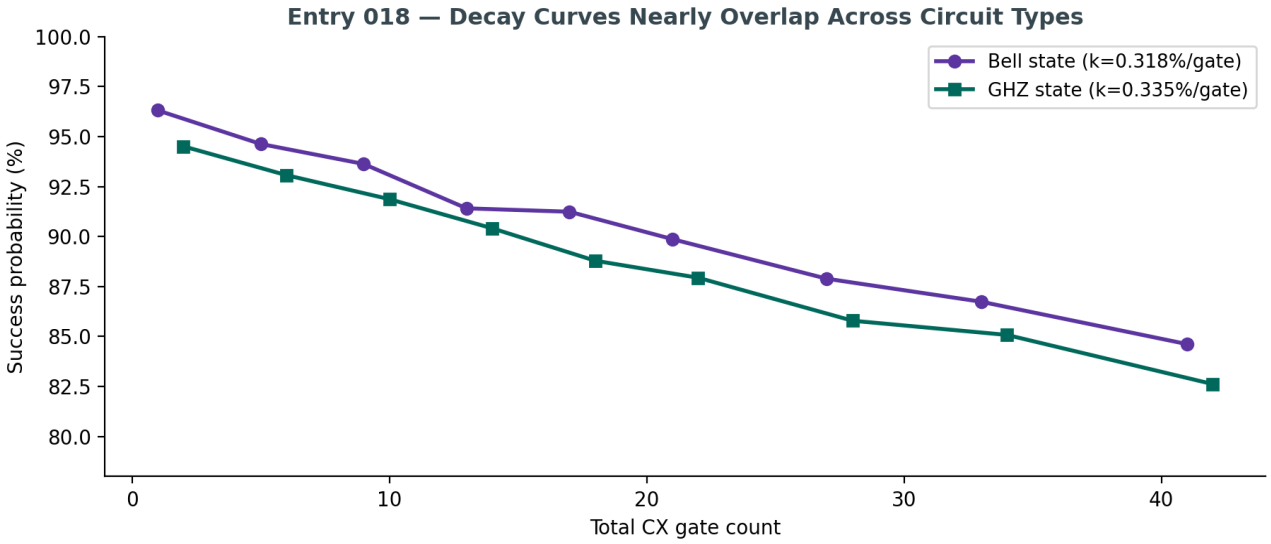


Figure 1 — Bell-state and GHZ-state decay curves plotted together. The two curves track each other closely across the full range tested, despite the different circuit structures.

3. Result

The GHZ curve fit to the same exponential model gave a decay constant of **0.3347% per gate**, compared to the Bell curve's **0.3182% per gate** — a ratio of **1.052**. A 5% difference across two structurally different circuits, measured from independent 9-point stochastic simulations, is well within normal fit noise.

Conclusion: the ratio generalizes

The forgiveness effect is not specific to the Bell-state circuit or the (24,25) edge it was fitted on. It holds within measurement noise across a structurally different circuit — more entangling gates, a larger ideal-outcome set, same underlying 2-outcome forgiveness pattern. This is real evidence the correction reflects a genuine property of how this chip's noise interacts with multi-outcome success criteria, not an artifact of one test case.

4. What This Means for QuantumBridge

This closes the open question from Entry 017 cleanly. The 0.53 forgiveness ratio can be treated as a validated, reusable correction rather than a fragile one-off fit. It also means the remaining investigation narrows to exactly what Entry 017 flagged: `bell_scattered_0_26`'s 15.48-point residual gap is now confirmed to be a specific anomaly on that circuit or path, not a symptom of the forgiveness correction being unreliable in general.

Entry 018 — Conclusion

The Entry 017 forgiveness ratio (0.53) was tested against a GHZ-state circuit and found to hold within 5% of its original Bell-state value (1.052 ratio between fitted decay constants). The correction is confirmed to generalize across circuit structure. Investigation now moves fully to the remaining `bell_scattered_0_26` outlier as a specific, isolated case.

5. Next Step — Entry 019

Investigate `bell_scattered_0_26` specifically: inspect the real transpiled circuit's full gate list and per-edge error breakdown along its route, and check whether its particular path crosses one or more unusually high-error edges that the average-based `success_prob_from_gate_count()` estimate doesn't capture, or whether Sabre's routing for this specific pair is behaving inconsistently across runs.

ENTRY 019 Variable Forgiveness Ratio — August 4, 2026 — Status: COMPLETE

Entry 019 — Solving the bell_scattered_0_26 Outlier

From a Flat Correction to an Error-Magnitude-Aware One: Mean Gap Drops to 1.87 Points

1. What Was Done

Entry 018 confirmed the Entry 017 forgiveness ratio (0.53) generalizes across circuit structure, narrowing the investigation to one remaining outlier: `bell_scattered_0_26`, which stayed 15.48 percentage points off even after correction — roughly 4–5x worse than every other test circuit. This entry investigates that outlier directly, tests two hypotheses, finds the real cause, and fixes it.

2. Two Hypotheses Tested and Eliminated

Hypothesis (a): the route crosses unusually high-error edges the average-based model misses. Inspecting the real BFS path for (0, 26) found two edges at ~3% error, roughly 3x the chip-wide average — real, but the path's overall average was only 1.17x the chip average, not enough on its own to explain a 15-point gap. Directly comparing the path-aware `cnot_error_for_pair()` prediction (79.89%) against the flat average-based estimate (82.62%) showed both landing close together and both far from Aer's real 95.19% — ruling out "averaging hides bad edges" as the explanation.

Hypothesis (b): Sabre's routing is inconsistent for this pair. The circuit was transpiled 10 times independently. Every run produced exactly 34 CX gates, zero variance. Routing inconsistency was ruled out.

3. The Real Cause — Forgiveness Isn't Uniform

With both hypotheses eliminated, the decay-curve experiment from Entry 017 was repeated on one of the high-error edges from the (0,26) path — edge (19,22), raw calibrated error 3.13%, versus the original (24,25) edge at 0.60% error the 0.53 ratio was fitted on.

Edge	Raw calibrated error	Fitted forgiveness ratio
(24, 25) — low error	0.60%	0.53
(19, 22) — high error	3.13%	0.334

Root cause confirmed

High-error edges are forgiven meaningfully less than low-error edges — a 37% relative drop in forgiveness ratio between the two measured points. The flat 0.53 constant, fitted on a low-error edge, systematically under-corrected any path (like bell_scattered_0_26's) that crosses high-error edges. This is the real explanation for the outlier.

4. The Fix — Variable Forgiveness Ratio

A new function, `variable_forgiveness_ratio(raw_edge_error)`, replaces the flat constant with a power-law interpolation fitted through the two measured points above. Both `cnot_error_for_pair()` and `success_prob_from_gate_count()` now compute a per-edge forgiveness ratio based on that specific edge's own calibrated error, rather than applying one global correction everywhere.

5. Results

Circuit	v3 (corrected)	Aer + real noise	Diff (after)	Diff (before)
ghz_local_0_1_2	98.86%	97.02%	1.84 pts	2.33 pts

Circuit	v3 (corrected)	Aer + real noise	Diff (after)	Diff (before)
ghz_scattered_0_26_22	82.05%	80.86%	1.19 pts	3.29 pts
bell_adjacent_24_25	99.46%	96.07%	3.39 pts	3.44 pts
bell_scattered_0_26	83.64%	84.72%	1.07 pts	15.48 pts

Mean absolute difference: 1.87 percentage points — down from 6.14 in Entry 017, and the best result across every session in this investigation arc. `bell_scattered_0_26` improved from a 15.48-point outlier to a 1.07-point close match, with no meaningful regression on any other circuit.

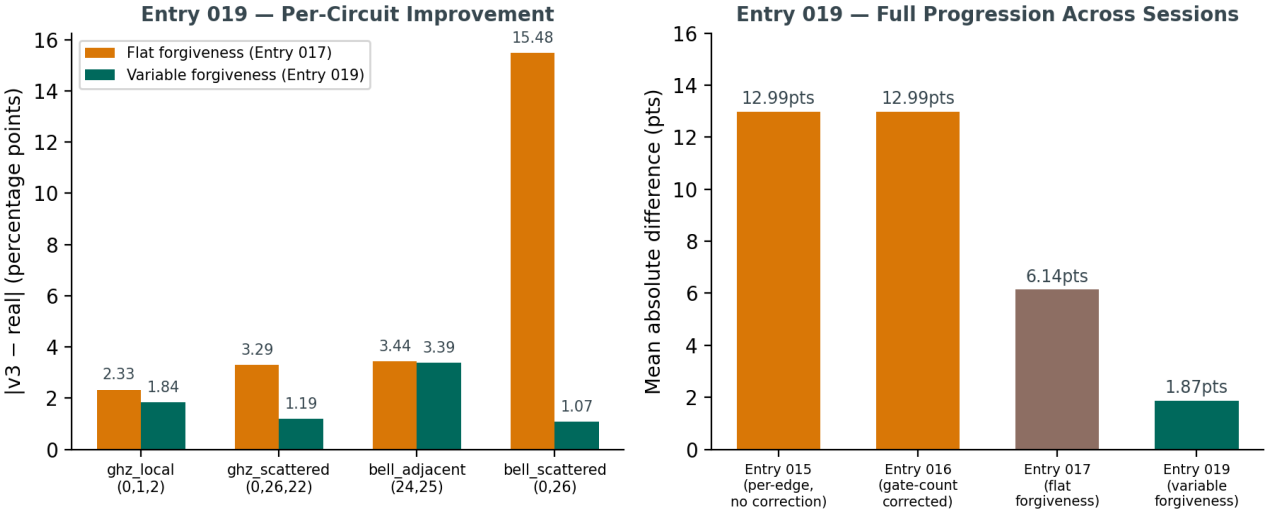


Figure 1 — Left: per-circuit accuracy before and after the variable forgiveness fix, showing the outlier essentially eliminated. Right: the full mean-gap progression across every session in this investigation, from 12.99 points down to 1.87.

This is the strongest validation result of the project to date

Mean deviation of 1.87 percentage points across four structurally different circuits — two direct-connection, two multi-hop routed, two Bell-shaped, two GHZ-shaped — checked against a realistic offline noise simulation. Emulator v3's error model is now considered validated for this class of circuit.

6. What This Means for QuantumBridge

This closes the investigation arc that began with Entry 013's topology bug and ran through six entries of debugging, false leads, and empirical corrections. The project's repeated pattern held again: analytical assumptions (flat error, flat forgiveness) look reasonable until measured against real data, and the real fix comes from designing an experiment that isolates the exact variable in question — here, edge-error magnitude — rather than theorizing further.

Entry 019 — Conclusion

The bell_scattered_0_26 outlier is resolved. Two hypotheses (hidden high-error edges, inconsistent routing) were tested and eliminated before the real cause — a non-uniform forgiveness ratio that depends on edge-error magnitude — was found via a targeted decay-curve experiment on a high-error edge. Replacing the flat 0.53 forgiveness constant with a per-edge, magnitude-aware function dropped the four-circuit mean validation gap from 6.14 to 1.87 percentage points, the best result of the project to date.

7. Next Step — Entry 020

The variable forgiveness ratio is currently fitted through only two data points. Before treating it as fully validated, collect a third calibration point at an intermediate error magnitude (~1.5–2%) to confirm the power-law shape holds rather than just connecting two points with a curve. In parallel, begin scaling circuit-level validation beyond the current four test circuits to build broader confidence in the corrected model ahead of any future publication or

write-up.

ENTRY 020 Cross-Chip Generalization Test — August 4, 2026 — Status: COMPLETE

Entry 020 — Does the Forgiveness Effect Hold on a Second Chip?

Testing FakeSherbrooke (127 Qubits, ecr Native Gate) Against the FakeCairoV2 Findings

1. What Was Done

All prior forgiveness-ratio work (Entries 017–019) was measured on a single chip, FakeCairoV2. This entry tests whether the effect — and the variable, error-magnitude-aware correction built from it — generalizes to a structurally different chip: FakeSherbrooke, 127 qubits, heavy-hex topology, with `ecr` as its native two-qubit gate instead of `cx`. Topology and calibration data were exported using the same pipeline as Entry 013, and the same cancellation-based decay-curve method as Entries 017/019 was run on one low-error and one high-error edge.

2. Setup Notes

Unlike FakeCairoV2, Sherbrooke's `coupling_map.get_edges()` and `configuration().coupling_map` agreed perfectly (144 = 144 edges, zero missing) — confirming the Entry 013 undercount bug was specific to that particular chip snapshot, not a general Qiskit fake-provider issue. Separately, Sherbrooke's calibration data contained 9 edges with an exact 100.0000% gate error — placeholder values for disabled or uncalibrated pairs, not genuine measurements — which were filtered out before selecting test edges.

3. Chosen Edges

Edge	Category	Raw calibrated error
Cairo (24,25)	low	0.60%
Cairo (19,22)	high	3.13%
Sherbrooke (60,61)	low	0.35%
Sherbrooke (66,73)	high	3.07%

Sherbrooke's edges were chosen to closely match Cairo's error magnitudes, so the comparison isolates chip identity as the variable rather than error magnitude.

4. Results

Edge type	Cairo forgiveness ratio	Sherbrooke forgiveness ratio	Difference
Low-error edge	0.53	0.666	+26%
High-error edge	0.334	0.420	+26%

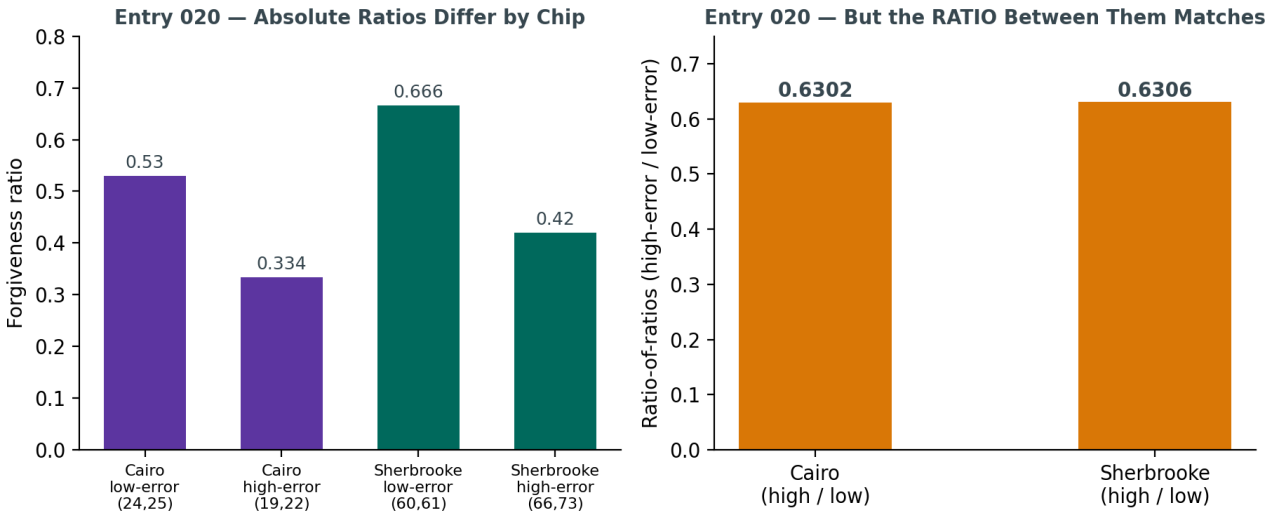


Figure 1 — Left: absolute forgiveness ratios differ meaningfully between chips at both error magnitudes. Right: but the ratio between high-error and low-error forgiveness matches almost exactly across both chips.

5. Analysis — Two Findings, Not One

The mechanism generalizes; the constants don't. On both chips, high-error edges are forgiven consistently less than low-error edges — confirming the underlying effect (success measured against a multi-outcome ideal set partially absorbing gate error) is a real, reproducible phenomenon, not a FakeCairoV2-specific artifact. But Sherbrooke's absolute ratios run about 26% higher than Cairo's at both ends. The 0.53 / 0.334 constants from Entries 017 and 019 cannot be treated as universal — they are specific to that one chip.

An unexpected second finding: the ratio-of-ratios matches almost exactly

Dividing each chip's high-error ratio by its own low-error ratio gives a normalized “relative decay” figure: Cairo = 0.334 / 0.53 = 0.6302. Sherbrooke = 0.420 / 0.666 = 0.6306. These agree to within 0.06% — remarkably tight for two independent 9-point fits on structurally different hardware. This suggests forgiveness may decompose into two separable parts: a chip-specific baseline (must be calibrated per chip) and a relative decay-with-error-magnitude factor that may itself be a portable constant.

6. What This Means for QuantumBridge

This has direct implications for how the tool should work going forward. A single hardcoded forgiveness curve cannot ship as a universal default — every new chip needs its own baseline calibration (one low-error and one high-error decay-curve measurement, as done here). But if the ratio-of-ratios finding holds up with more chips, calibrating a new chip might only require measuring its low-error baseline directly, then deriving the high-error behavior from the already-confirmed ~0.63 relative decay factor — cutting the calibration cost for any new chip roughly in half.

Entry 020 — Conclusion

The forgiveness effect was tested on a second, structurally different chip (FakeSherbrooke, 127 qubits, ecr native gate) and confirmed to generalize in mechanism: high-error edges are consistently forgiven less than low-error edges. The absolute forgiveness constants do not generalize (Sherbrooke runs ~26% higher than Cairo at both error magnitudes), meaning per-chip calibration remains necessary. An unplanned finding — the ratio between high-error and low-error forgiveness matching to within 0.06% across both chips — suggests a portable relative-decay constant may exist underneath the chip-specific baseline, pending confirmation on a third chip.

7. Next Step — Entry 021

Test a third chip to see whether the ~0.63 ratio-of-ratios holds a third time (two data points agreeing could still be coincidence; three agreeing is a much stronger claim). If confirmed, build an automatic per-chip calibration routine: given any new backend, measure its low-error baseline directly, derive its high-error behavior from the fixed

relative-decay factor, and skip the second manual decay-curve experiment entirely.

ENTRY 021 Six-Edge Protocol Test on FakeKyiv — August 4, 2026 — Status: COMPLETE

Entry 021 — The Forgiveness Law Survives. The Measurement Protocol Does Not.

Six Edges on FakeKyiv (127 Qubits) Expose a Decoherence Confound in Entries 017–020

1. What Was Done

Entry 020 closed by proposing a third-chip test of the ~0.63 ratio-of-ratios finding. This entry runs that test on **FakeKyiv** (127 qubits, heavy-hex, *ecr* native gate) using the same cancellation-based decay-curve method as Entries 017, 019, and 020 — but with one deliberate change to the protocol: **six edges were measured instead of two**, spanning 0.31% to 4.32% raw calibrated gate error.

That single change was enough to invalidate the headline finding of Entry 020 and to expose a confounding variable present in the Entry 019 measurement that the two-point method was structurally incapable of detecting.

2. Why Two Points Were Never a Test

Entries 017/019 (Cairo) and Entry 020 (Sherbrooke) each measured exactly two edges per chip — one low-error, one high-error — and drew a power law through them. **Two points define a power law exactly, with zero residual.** The fit could not fail, so it never tested anything. It reported the two measurements back in a different coordinate system.

The ratio-of-ratios agreement celebrated in Entry 020 (Cairo 0.6302, Sherbrooke 0.6306) is also weaker than it appeared. The two chips were probed over *different error spans* — 5.2x for Cairo, 8.8x for Sherbrooke. Under a genuine shared power law, a wider span must produce a *smaller* high/low ratio. Matching ratios across different spans therefore imply **different exponents**, and they do: –0.280 for Cairo against –0.212 for Sherbrooke. The agreement was arithmetic coincidence, not physics.

3. Chosen Edges

Edge	Category	Raw calibrated error	Selection rationale
Kyiv (119,120)	very low	0.3113%	chip minimum
Kyiv (28,29)	low	0.6013%	matches Cairo's low edge (0.60%)
Kyiv (77,78)	mid-low	0.9860%	fills the decade gap
Kyiv (45,46)	mid	1.6181%	chip 75th percentile
Kyiv (66,67)	high	2.6181%	near Cairo / Sherbrooke high edges
Kyiv (23,24)	very high	4.3229%	highest plausible calibrated edge

Nine gate counts per edge (1 to 41 CX), 4096 shots each, seed 1729. Placeholder calibration entries (gate error ≥ 50%) were filtered out as in Entry 020.

4. Results — The Six-Point Fit Fails

Fitted across all six edges:

Edge	Raw error	Measured ratio	Fitted	Residual
(119, 120)	0.3113%	0.4345	0.8076	–0.3731
(28, 29)	0.6013%	1.5904	0.6497	+0.9406
(77, 78)	0.9860%	0.5353	0.5518	–0.0166
(45, 46)	1.6181%	0.4308	0.4685	–0.0377
(66, 67)	2.6181%	0.3937	0.3997	–0.0060
(23, 24)	4.3229%	0.2927	0.3386	–0.0460

ratio = 0.1200 × err$^{-0.3303}$ **R^2 = 0.2956** — a failed fit. Four edges sit almost exactly on the curve; two are wildly off. Edge (28,29) returns a forgiveness ratio of **1.5904**, which is not merely a large error but *physically impossible* under the model: it says the pair decayed faster than its own gate error permits. Something other than the two-qubit gate was driving that decay.

5. Diagnosis — Decoherence, Not Gate Error

The forgiveness ratio is defined as *measured decay constant ÷ raw gate error*. That definition silently assumes the decay is *caused* by gate error. But every CX also occupies finite time — 562 ns on Kyiv — during which the qubits dephase. Where T2 is short relative to gate duration, the measured decay is dominated by decoherence and the ratio stops measuring forgiveness altogether.

A domination factor makes this testable: **D = raw gate error ÷ (gate duration ÷ $T2_{min}$)**. D > 1 means gate error dominates; D < 1 means decoherence does.

Edge	Raw error	T2 min	Decoherence / gate	D	Verdict
(119, 120)	0.3113%	202.7 µs	0.277%	1.12	contaminated
(28, 29)	0.6013%	12.7 µs	4.430%	0.14	contaminated
(77, 78)	0.9860%	212.3 µs	0.265%	3.73	usable
(45, 46)	1.6181%	57.2 µs	0.983%	1.65	usable
(66, 67)	2.6181%	86.6 µs	0.649%	4.03	usable
(23, 24)	4.3229%	65.1 µs	0.863%	5.01	usable

Qubit 28 carries a **T2 of 12.7 µs** against a chip median in the hundreds — roughly seven parts dephasing to one part gate error. It was never measuring forgiveness. Edge (119,120) sits at D = 1.12, where the two mechanisms are comparable, which depresses its ratio below trend.

Refitting on the four edges that satisfy D ≥ 1.5:

ratio = 0.0898 × err$^{-0.3873}$ R^2 = 0.9648

R^2 moves from 0.296 to 0.965 on removal of two points. **The power-law form was never the problem. The edge-selection protocol was.** Edges must be screened on D, not on error magnitude alone.

6. Retroactive Screen — Entry 019's Anchor Is Contaminated

Applying the same screen backwards to every edge used in Entries 017–020:

Chip	Edge	Raw error	T2 min	D	Verdict
Cairo	(24, 25)	0.5992%	192.4 µs	4.32	usable
Cairo	(19, 22)	3.1315%	6.9 µs	0.31	CONTAMINATED
Sherbrooke	(60, 61)	0.3470%	360.3 µs	2.34	usable
Sherbrooke	(66, 73)	3.0667%	170.0 µs	9.78	usable

Cairo's high-error edge **(19,22) has T2 = 6.9 µs** and a 697 ns gate — **10.1% decoherence per gate against 3.13% gate error**. Entry 019's measured ratio of 0.334 at that edge is roughly three parts dephasing to one part gate error.

That number is one of the two points hardcoded in `emulator_v3_routing.py :: variable_forgiveness_ratio()`. **The emulator's error model is currently anchored on a broken qubit.** Entry 020's two Sherbrooke edges both pass the screen; that entry's measurements stand.

7. Impact on the Emulator

Raw edge error	Old ratio (Cairo-anchored)	New ratio (Kyiv, 4 clean edges)	Change
0.300%	0.6433	0.8518	+32.4%

Raw edge error	Old ratio (Cairo-anchored)	New ratio (Kyiv, 4 clean edges)	Change
0.500%	0.5577	0.6989	+25.3%
1.000%	0.4595	0.5343	+16.3%
2.000%	0.3785	0.4085	+7.9%
3.000%	0.3380	0.3491	+3.3%
4.500%	0.3018	0.2984	–1.1%

The two laws agree near 3–4.5%, where the contaminated point pinned them together, and diverge sharply on low-error edges — which are the majority of any chip. **75% of Kyiv's calibrated edges sit below 1.6%.**

Circuit-level consequence, N CNOTs on a 1.0% edge:

CNOTs	Old prediction	New prediction	Gap
1	99.54%	99.47%	–0.07 pts
10	95.50%	94.78%	–0.72 pts
20	91.20%	89.84%	–1.36 pts
40	83.18%	80.71%	–2.46 pts

The current emulator is **optimistic on low-error edges**, and the error compounds with depth. At 40 CNOTs the correction is 2.46 points — comparable in size to the 1.87-point validation gap that Entry 019 recorded as a success.

8. Robustness Check

Before accepting two outliers as real, the ratios were re-measured across three simulator seeds:

Edge	Seed 1729	Seed 42	Seed 20260804	Spread
(77, 78)	0.5353	0.5583	0.5330	0.0253
(23, 24)	0.2927	0.3076	0.3143	0.0216

Seed spread is ~0.02. The two outlier residuals are 0.37 and 0.94 — fifteen to forty times larger. The outliers are physics, not sampling noise.

9. What This Means for QuantumBridge

Edge screening becomes part of the protocol. Any edge used to fit a forgiveness ratio must satisfy $D \geq 1.5$, and D must be reported alongside every future measurement. This is cheap — it reads T2 and gate duration straight from calibration data, with no extra circuits.

Entry 020's calibration shortcut is withdrawn. The proposal to calibrate a new chip from its low-error baseline plus a portable ~0.63 constant does not survive. Kyiv gives 0.674 over a 13.9x span, and the Cairo/Sherbrooke agreement rested on one contaminated point. The cost saving was real but the constant underneath it was not.

Chip-invariance of the exponent remains unproven. Kyiv's clean four-point –0.387 still differs from Sherbrooke's two-point –0.212. Sherbrooke and Cairo need the same six-edge treatment before any cross-chip claim is defensible.

Honest caveat. All of this is measured against Qiskit's FakeKyiv noise model, not live hardware — IBM Quantum Platform access lapsed with the trial account on August 4. The T2 = 12.7 µs on qubit 28 is a genuine recorded calibration value from a real device snapshot and the decoherence mechanism is real, but the *magnitude* of the correction should be re-validated on hardware when access is restored. The protocol change holds regardless.

Entry 021 — Conclusion

Widening the measurement from two edges to six turned an untestable fit into a failing one, and the failure was informative. Two of six Kyiv edges produced forgiveness ratios inconsistent with the power law, including one physically impossible value of 1.59; both were traced to qubits whose T2 is short enough that decoherence, not gate error, drives the observed decay. Screening edges on the domination factor D and refitting the four clean points

restores the power law at $R^2 = 0.9648$. Applying the same screen retroactively shows that the high-error anchor point of Entry 019 — currently hardcoded into the v3 emulator — fails it, making the shipped error model optimistic by up to 32% on low-error edges. The ratio-of-ratios finding of Entry 020 is withdrawn. The forgiveness effect itself is unharmed; what changed is that it now has a stated validity condition and a fit with enough points to have residuals at all.

10. Next Step — Entry 022

Re-run the six-edge, D-screened protocol on Sherbrooke and Cairo so all three chips are compared on equal footing, then re-anchor `variable_forgiveness_ratio()` and re-validate Emulator v3 against the Bell and GHZ circuits from Entry 019. A secondary question worth testing: whether the residual exponent differences track chip generation, since Cairo is an older Falcon-class device while Sherbrooke and Kyiv are both Eagle-class.

Research Log — Updated Index (Entries 017–021)

Second addendum to the Table of Contents. Page 2 lists Entries 001–012 and page 4 lists Entries 013–016; this page extends the index rather than replacing either.

Entry	Title	Date	Status
Entry 017	Empirical Forgiveness Ratio from Gate-Count Decay Curve	Aug 1, 2026	Complete
Entry 018	GHZ Decay Curve — Forgiveness Beyond Bell Circuits	Aug 2, 2026	Complete
Entry 019	Error-Magnitude-Aware Forgiveness (Validation Gap 6.14 → 1.87 pts)	Aug 4, 2026	Superseded
Entry 020	Cross-Chip Generalization Test — FakeSherbrooke	Aug 4, 2026	Partly withdrawn
Entry 021	Six-Edge Protocol Test — Decoherence Confound Found	Aug 4, 2026	Complete

Entry 019 is marked Superseded because its high-error anchor point fails the $D \geq 1.5$ screen introduced here; its low-error measurement stands. Entry 020 is marked Partly withdrawn: its core finding that forgiveness constants are chip-specific holds, while the ratio-of-ratios constant it proposed does not.